

**SKRIPSI**  
**KLASIFIKASI PENYAKIT BAWANG MERAH DENGAN**  
**PENDEKATAN DEEP LEARNING**



**NATASHA**  
**NIM. 222280055**

**PROGRAM STUDI TEKNIK INFORMATIKA**  
**FAKULTAS TEKNIK**  
**UNIVERSITAS MUHAMMADIYAH PAREPARE**  
**2026**

## **HALAMAN PERSETUJUAN**

### **KLASIFIKASI PENYAKIT BAWANG MERAH DENGAN PENDEKATAN DEEP LEARNING**

**NATASHA**

**NIM. 222280055**

Telah diperiksa dan disetujui untuk mengikuti ujian tutup

Parepare, 19 Agustus 2026  
Komisi Pembimbing

Pembimbing I

Pembimbing II

**Mughaffir Yunus, ST., MT**  
**NBM. 1252 809**

**Hasnawati, S.Kom., M.Kom**  
**NBM. 1266 453**

Mengetahui:

Plt. Ketua Program Studi  
Teknik Informatika

**Mughaffir Yunus, ST., MT**  
**NBM. 1252 809**

## PRAKATA

*Bismillahirrahmanirahim*

*Alhamdulillahillobbil'alamin.* Puji syukur kehadiran Allah *subhanahu wa ta'ala* atas rahmat dan hidayah-Nya, sehingga atas izin-Nya pula dapat menyusun proposal penelitian ini dengan judul **“Klasifikasi Penyakit Bawang Merah Dengan Pendekatan Deep Learning”**.

Penelitian ini dilatar belakangi oleh pentingnya bawang merah sebagai komoditas pertanian yang memiliki ekonomi tinggi dan peran strategi dalam ketahanan pangan. Namun, serangan penyakit pada tanaman bawang merah seringkali mengakibatkan kerugian yang signifikan bagi para petani. Dalam era digital ini, teknologi deep learning menawarkan solusi inovatif dan efisien dalam menganalisis dan mengklasifikasikan data. Dengan memanfaatkan metode ini, kami berharap dapat mengembangkan sistem yang mampu mendeteksi dan mengklasifikasikan penyakit bawang merah secara akurat, sehingga dapat membantu petani dalam mengambil langkah pencegahan yang tepat dan meningkatkan hasil panen.

Proposal ini disusun dengan harapan dapat memberikan kontribusi yang berarti bagi pengembangan ilmu pengetahuan, khususnya dalam bidang pertanian dan teknologi informasi. Penulis menyadari bahwa penelitian ini tidak akan terwujud tanpa dukungan dan bimbingan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada semua pihak yang telah memberikan dukungan, baik secara langsung maupun tidak langsung.

Dengan menyadari bahwa kesempurnaan hanyalah milik Allah Subhanahu Wa Ta'ala, penulis dengan segala kerendahan hati memohon kritik dan saran yang membangun dari para pembaca. Semoga proposal penelitian ini dapat memberikan manfaat bagi perkembangan ilmu pengetahuan dan teknologi serta menjadi referensi bagi penelitian di bidang pertanian.

*Nashrumminallah wafathun qarib.*

Parepare, 16 Juni 2026  
Penulis

**NATASHA**  
**NIM. 222280055**

## **ABSTRAK**

**NATASHA** “Klasifikasi Penyakit Bawang Merah dengan Pendekatan Deep Learning”. (Dibimbing oleh Mughaffir Yunus dan Hasnawati)

**Kata Kunci:**

## ***ABSTRACT***

**NATASHA** *"Classification of Shallot Diseases using Deep Learning Approach".  
(Supervised by Mughaffir Yunus and Hasnawati)*

**Keywords:**

## DAFTAR ISI

|                          | Halaman |
|--------------------------|---------|
| HALAMAN JUDUL            | i       |
| HALAMAN PERSETUJUAN      | ii      |
| PRAKATA                  | iii     |
| ABSTRAK                  | v       |
| <i>ABSTRACT</i>          | vi      |
| DAFTAR ISI               | vii     |
| DAFTAR TABEL             | xi      |
| DAFTAR GAMBAR            | xiii    |
| BAB I PENDAHULUAN        | 1       |
| A. Latar Belakang        | 1       |
| B. Rumusan Masalah       | 2       |
| C. Tujuan Penelitian     | 3       |
| D. Batasan Masalah       | 3       |
| E. Manfaat Penelitian    | 4       |
| F. Sistematika Penulisan | 5       |
| BAB II TINJAUAN PUSTAKA  | 7       |
| A. Kajian Teori          | 7       |
| 1. Penyakit Bawang Merah | 7       |

|                           |                                           |    |
|---------------------------|-------------------------------------------|----|
| 2.                        | Citra Digital                             | 10 |
| 3.                        | <i>Deep Learning</i>                      | 11 |
| 4.                        | <i>Convolutional Neural Network (CNN)</i> | 12 |
| 5.                        | <i>Confussion Matriks</i>                 | 17 |
| 6.                        | <i>Visual Studio Code</i>                 | 19 |
| 7.                        | <i>Python</i>                             | 20 |
| 8.                        | HTML ( <i>HyperText Markup Language</i> ) | 24 |
| 9.                        | CSS                                       | 25 |
| 10.                       | JavaScript                                | 26 |
| 11.                       | Flask                                     | 27 |
| 10.                       | <i>Unified Modeling Language (UML)</i>    | 28 |
| BAB III METODE PENELITIAN |                                           | 38 |
| A.                        | Jenis Penelitian                          | 38 |
| B.                        | Lokasi dan Waktu                          | 38 |
| C.                        | Alat dan Bahan                            | 39 |
| 1.                        | Alat                                      | 39 |
| 2.                        | Bahan                                     | 40 |
| D.                        | Rancangan Sistem                          | 41 |
| 1.                        | Use Case Diagram                          | 41 |
| 2.                        | Desain UI yang diusulkan                  | 42 |



|                                            |                                     |
|--------------------------------------------|-------------------------------------|
| E. Teknik Pengumpulan Data                 | 46                                  |
| F. Teknik Analisis Data                    | 48                                  |
| <b>BAB IV HASIL DAN PEMBAHASAN</b>         | <b>50</b>                           |
| A. Analisis Kebutuhan dan Rancangan        | <b>Error! Bookmark not defined.</b> |
| 1. Flowchart Sistem                        | <b>Error! Bookmark not defined.</b> |
| 2. Alur Kerja CNN                          | <b>Error! Bookmark not defined.</b> |
| B. Implementasi <i>Tools</i> Pengembangan  | <b>Error! Bookmark not defined.</b> |
| 1. Python Flask                            | <b>Error! Bookmark not defined.</b> |
| 2. <i>TensorFlow</i> Keras                 | <b>Error! Bookmark not defined.</b> |
| 3. Numpy                                   | <b>Error! Bookmark not defined.</b> |
| 4. SQLite                                  | <b>Error! Bookmark not defined.</b> |
| C. Explore Dataset                         | <b>Error! Bookmark not defined.</b> |
| 1. Deskripsi Dataset                       | <b>Error! Bookmark not defined.</b> |
| 2. Deskripsi Data Penelitian               | <b>Error! Bookmark not defined.</b> |
| D. Evaluasi Performa Model                 | <b>Error! Bookmark not defined.</b> |
| 1. <i>Plotting</i> Akurasi dan <i>Loss</i> | <b>Error! Bookmark not defined.</b> |
| 2. <i>Confusion Matrix</i>                 | <b>Error! Bookmark not defined.</b> |
| 3. Perhitungan Manual CNN                  | <b>Error! Bookmark not defined.</b> |
| E. Implementasi Sistem                     | <b>Error! Bookmark not defined.</b> |
| 1. Halaman Beranda                         | <b>Error! Bookmark not defined.</b> |

|                                   |                                     |
|-----------------------------------|-------------------------------------|
| 2. Halaman Input                  | <b>Error! Bookmark not defined.</b> |
| 3. Halaman Konvolusi              | <b>Error! Bookmark not defined.</b> |
| 4. Halaman ReLU                   | <b>Error! Bookmark not defined.</b> |
| 5. Halaman Pooling                | <b>Error! Bookmark not defined.</b> |
| 6. Halaman Global Average Pooling | <b>Error! Bookmark not defined.</b> |
| 7. Halaman Fully Connected Layer  | <b>Error! Bookmark not defined.</b> |
| 8. Halaman Hasil Deteksi          | <b>Error! Bookmark not defined.</b> |
| F. Pengujian Sistem               | <b>Error! Bookmark not defined.</b> |
| 1. Pengujian <i>Black Box</i>     | <b>Error! Bookmark not defined.</b> |
| 2. Hasil Pelatihan Model          | <b>Error! Bookmark not defined.</b> |
| <b>BAB V PENUTUP</b>              | <b>50</b>                           |
| A. Kesimpulan                     | <b>Error! Bookmark not defined.</b> |
| B. Saran                          | <b>Error! Bookmark not defined.</b> |
| <b>DAFTAR PUSTAKA</b>             | <b>74</b>                           |

## DAFTAR TABEL

|                                                                                                | Halaman                             |
|------------------------------------------------------------------------------------------------|-------------------------------------|
| <b>Tabel 2. 1</b> Simbol Flowchart umum digunakan                                              | 29                                  |
| <b>Tabel 2. 2</b> Simbol Use Case Diagram                                                      | 30                                  |
| <b>Tabel 2. 3</b> Simbol <i>Activity Diagram</i>                                               | 31                                  |
| <b>Tabel 2. 4</b> Simbol <i>Sequence Diagram</i>                                               | 32                                  |
| <b>Tabel 2. 5</b> Contoh Pengujian Black – Box Testing                                         | 34                                  |
| <b>Tabel 3. 1</b> Lokasi dan Waktu                                                             | 38                                  |
| <b>Tabel 3. 2</b> Deskripsi Use Case Diagram                                                   | 41                                  |
| <b>Tabel 4. 1</b> Distribusi Dataset                                                           | <b>Error! Bookmark not defined.</b> |
| <b>Tabel 4. 2</b> Prediksi benar pada dataset                                                  | <b>Error! Bookmark not defined.</b> |
| <b>Tabel 4. 3</b> Kesalahan deteksi pada test dataset                                          | <b>Error! Bookmark not defined.</b> |
| <b>Tabel 4. 4</b> Precision, Recall, dan F1-score                                              | <b>Error! Bookmark not defined.</b> |
| <b>Tabel 4. 5</b> Sampel Kalkulasi Kontribusi Fitur GAP terhadap Neuron Fully Connected Layer. | <b>Error! Bookmark not defined.</b> |
| <b>Tabel 4. 6</b> Black Box Halaman Beranda                                                    | <b>Error! Bookmark not defined.</b> |
| <b>Tabel 4. 7</b> Black Box Halaman Input                                                      | <b>Error! Bookmark not defined.</b> |
| <b>Tabel 4. 8</b> Black Box Halaman Konvolusi                                                  | <b>Error! Bookmark not defined.</b> |
| <b>Tabel 4. 9</b> Black Box Halaman ReLU                                                       | <b>Error! Bookmark not defined.</b> |
| <b>Tabel 4. 10</b> Black Box Halaman pooling                                                   | <b>Error! Bookmark not defined.</b> |
| <b>Tabel 4. 11</b> Black Box Halaman Global Average Pooling                                    | <b>Error! Bookmark not defined.</b> |

**Tabel 4. 12** Black Box Halaman Fully Connected      **Error! Bookmark not defined.**

**Tabel 4. 13** Black Box Halaman Hasil Deteksi      **Error! Bookmark not defined.**

**Tabel 4. 14** Hyperparameter pelatihan model      **Error! Bookmark not defined.**

**Tabel 4. 15** Riwayat Pelatihan Model      **Error! Bookmark not defined.**

## DAFTAR GAMBAR

|                                                           | Halaman                             |
|-----------------------------------------------------------|-------------------------------------|
| <b>Gambar 2. 1</b> Penyakit Trotoir Pada Bawang Merah     | 7                                   |
| <b>Gambar 2. 2</b> Penyakit Moler Pada Bawang Merah       | 8                                   |
| <b>Gambar 2. 3</b> Alur Pengolahan Citra                  | 10                                  |
| <b>Gambar 2. 4</b> Alur Convolutional Neural Network      | 12                                  |
| <b>Gambar 2. 6</b> Visual Studio Code                     | 19                                  |
| <b>Gambar 2. 7</b> Python                                 | 20                                  |
| <b>Gambar 2. 8</b> HTML (Hypertext Markup Language)       | 24                                  |
| <b>Gambar 2. 9</b> Unified Modeling Language              | 28                                  |
| <b>Gambar 2. 10</b> Black-box Testing                     | 33                                  |
| <b>Gambar 2. 11</b> Kerangka pikir                        | 37                                  |
| <b>Gambar 3. 1</b> Use Case Diagram                       | 41                                  |
| <b>Gambar 3. 2</b> Tampilan Input Gambar                  | 42                                  |
| <b>Gambar 3. 3</b> Tampilan Lapisan Konvolusi             | 43                                  |
| <b>Gambar 3. 4</b> Tampilan Fungsi Aktivasi ReLU          | 43                                  |
| <b>Gambar 3. 5</b> Tampilan <i>Pooling Layer</i>          | 44                                  |
| <b>Gambar 3. 6</b> Tampilan <i>Global Average Pooling</i> | 44                                  |
| <b>Gambar 3. 7</b> Tampilan Fully Connected Layer         | 45                                  |
| <b>Gambar 3. 8</b> Tampilan hasil deteksi                 | 46                                  |
| <b>Gambar 4. 1</b> Flowchart Pengguna                     | <b>Error! Bookmark not defined.</b> |
| <b>Gambar 4. 2</b> Flowchart CNN                          | <b>Error! Bookmark not defined.</b> |
| <b>Gambar 4. 3</b> Grafik Akurasi dan Loss                | <b>Error! Bookmark not defined.</b> |

**Gambar 4. 4** Confussion Matrix Model                      **Error! Bookmark not defined.**

**Gambar 4. 5** Perhitungan nilai probabilitas dengan fungsi softmax                      **Error! Bookmark not defined.**

**Gambar 4. 6** Halaman Beranda                      **Error! Bookmark not defined.**

**Gambar 4. 7** Halaman Input                      **Error! Bookmark not defined.**

**Gambar 4. 8** Halaman Konvolusi                      **Error! Bookmark not defined.**

**Gambar 4. 9** Halaman ReLU                      **Error! Bookmark not defined.**

**Gambar 4. 10** Halaman Pooling                      **Error! Bookmark not defined.**

**Gambar 4. 11** Halaman Global Average Pooling                      **Error! Bookmark not defined.**

**Gambar 4. 12** Halaman Fully Connected                      **Error! Bookmark not defined.**

**Gambar 4. 13** Halaman Deteksi                      **Error! Bookmark not defined.**

# **BAB I**

## **PENDAHULUAN**

### **A. Latar Belakang**

Bawang merah (*Allium ascalonicum L.*) merupakan salah satu komoditas hortikultura strategis bernilai ekonomi tinggi yang menopang ketahanan pangan nasional. Berdasarkan data Badan Pusat Statistik (BPS) produksi nasional tahun 2024 mencapai mencapai 2,14 juta ton, dengan produktivitas rata-rata 10,74 ton per hektar di mana lebih dari 65% disuplai dari Jawa Tengah, Jawa Timur, dan Nusa Tenggara Barat (Sholihah, 2024). Khusus di wilayah Sulawesi Selatan, Kabupaten Enrekang berperan sebagai daerah penghasil utama bawang merah yang memegang peranan krusial dalam penyediaan pasokan regional, sehingga membutuhkan dukurngan teknologi informasi dan sistem pemantauan yang modern (Mubarak dkk., 2025). Namun, upaya peningkatan produksi ini kerap terhambat oleh serangan organisme pengganggu tanaman, khususnya penyakit bercak ungu yang disebabkan oleh *Alternaria Porri* dan penyakit layu fusarium yang disebabkan oleh *Fusarium oxysporum* yang menurunkan kualitas serta kuantitas hasil panen (Pongu Lima Manang & dan Astri Sumiati, 2024).

Selama ini, proses identifikasi penyakit masih bergantung pada pengamatan visual secara manual oleh petani. Metode konvensional tersebut dinilai kurang efisien, memakan waktu, serta rentan terhadap kekeliruan diagnosis yang berisiko memperparah kerugian ditingkat petani. Oleh karena itu, penerapan teknologi

pertanian presisi menjadi kebutuhan mendesak untuk menghadirkan sistem deteksi dini yang cepat, objektif dan akurat.

Perkembangan kecerdasan buatan menawarkan solusi melalui pemanfaatan *Deep Learning*, khususnya arsitektur *Convolutional Neural Network* (CNN) yang efektif untuk mengenali pola visual pada citra daun tanaman. Penelitian terdahulu juga menunjukkan bahwa model CNN mampu mengklasifikasikan penyakit tanaman dengan tingkat akurasi yang tinggi (Rijal dkk., 2024). Integrasi model ini dengan bahasa pemrograman Python serta pustaka pendukung seperti TensorFlow atau PyTorch memungkinkan pengembangan sistem cerdas yang handal dan aplikatif.

Penelitian ini mengambil kebaruan dengan merancang sistem klasifikasi penyakit daun bawang merah menggunakan arsitektur CNN standar berbasis Python. Pendekatan ini difokuskan untuk menghadirkan alternatif solusi yang lebih sederhana, efisien, dan aplikatif tanpa kompleksitas teknik *transfer learning* yang berlebihan, melalui perancangan ini, diharapkan tercipta sebuah landasan teknologi praktis yang dapat diintegrasikan ke dalam platform web guna mempermudah akses bagi petani maupun praktisi pertanian digital di Indonesia.

## **B. Rumusan Masalah**

Berdasarkan latar belakang yang sudah diuraikan, dapat disimpulkan bahwa rumusan masalah pada penelitian ini yaitu :

1. Bagaimana merancang sistem *Convolution Neural Network* (CNN) yang mampu mengidentifikasi dan mendeteksi penyakit trotoir dan molar pada bawang merah menggunakan python?



2. Bagaimana sistem *Convolution Neural Network* (CNN) berfungsi untuk membantu petani dalam mengklasifikasikan penyakit trotol dan moler?

### **C. Tujuan Penelitian**

Berdasarkan rumusan masalah yang sudah diuraikan, dapat disimpulkan bahwa tujuan pada penelitian ini yaitu :

1. Merancang dan mengembangkan sistem berbasis *Convolutional Neural Network* (CNN) yang mampu mengidentifikasi dan mendeteksi penyakit trotol dan moler pada bawang merah menggunakan bahasa pemrograman Python.
2. Mengimplementasikan sistem *Convolution Neural Network* (CNN) yang dapat membantu petani dalam mengklasifikasikan penyakit trotol dan moler secara cepat dan akurat.

### **D. Batasan Masalah**

Dalam penelitian ini ruang lingkup dibatasi secara spesifik untuk memastikan terfokus dan tidak meluas dari pembahasan. Adapun batasan-batasan masalah telah diterapkan meliputi :

1. Penelitian ini akan difokuskan untuk mengidentifikasi dan mengklasifikasikan penyakit trotol dan moler yang ditemukan pada tanaman bawang merah.
2. Penelitian ini akan menggunakan kecerdasan buatan dengan pendekatan *Deep Learning* dengan model *Convolution Neural Network* (CNN) untuk menganalisis citra daun bawang merah.
3. Jumlah dataset citra yang digunakan dalam penelitian ini yaitu 2000 gambar dengan format *.jpg/jpeg*.

4. *Dataset* terbagi menjadi 3 yaitu data *training* dengan *persentase* 80%, data *validation* 10%, dan data *testing* 10%.

### **E. Manfaat Penelitian**

Adapun manfaat yang dapat diberikan dari penelitian ini adalah sebagai berikut :

#### **1. Manfaat Bagi Penulis**

Bagi Penulis, penelitian ini menjadi kesempatan untuk mengaplikasikan ilmu pengetahuan dan teori yang telah diperoleh selama masa perkuliahan, khususnya dalam bidang kecerdasan buatan dan pengolahan citra digital, ke dalam bentuk karya ilmiah.

#### **2. Manfaat Bagi Institusi**

Penelitian ini diharapkan dapat menjadi bahan acuan, perbandingan, maupun landasan awal bagi mahasiswa yang tertarik mengembangkan penelitian serupa, terutama yang berfokus pada pemanfaatan teknologi kecerdasan buatan di sektor pertanian.

#### **3. Manfaat Bagi Petani Maupun Dinas Pertanian dan Penyuluh**

Website ini diharapkan memberikan solusi praktis bagi para petani dalam mengenali dan mengidentifikasi jenis penyakit pada tanaman bawang merah, seperti penyakit trotol dan moler secara lebih cepat, mandiri, dan akurat berdasarkan citra daun. Di sisi lain, bagi pihak Dinas Pertanian dan petugas penyuluh lapangan, hasil dalam penelitian ini dapat dimanfaatkan sebagai sarana pendukung dalam memberikan edukasi serta informasi yang tepat sasaran kepada masyarakat, sehingga tindakan pencegahan maupun penanganan dini

dapat dilakukan secara efektif untuk menekan tingkat gagal panen dan meningkatkan produktivitas pertanian.

## **F. Sistematika Penulisan**

### **BAB I PENDAHULUAN**

Memuat uraian mengenai latar belakang masalah, rumusan masalah, tujuan penelitian, batasan masalah, manfaat penelitian, serta sistematika penulisan. Bagian ini memaparkan permasalahan yang diangkat, alasan mendasar di balik pelaksanaan penelitian, serta arah tujuan dan kontribusi yang diharapkan.

### **BAB II TINJAUAN PUSTAKA**

Membahas landasan teori, kajian penelitian terdahulu, dan kerangka berpikir. Bagian ini berfungsi sebagai landasan teoritis serta tinjauan ilmiah yang memperkuat dasar pelaksanaan penelitian.

### **BAB III METODE PENELITIAN**

Memuat jenis penelitian, lokasi dan waktu penelitian, alat dan bahan, rancangan sistem, teknik pengumpulan data, teknik analisis data, serta diagram alir. Bagian ini menjelaskan prosedur metodologis secara runtut yang digunakan untuk mencapai tujuan penelitian.

### **BAB IV HASIL DAN PEMBAHASAN**

Memuat uraian hasil pengujian sistem, implementasi model *Deep Learning*, serta analisis mendalam terhadap hasil klasifikasi penyakit bawang merah.

## **BAB V PENUTUP**

Memuat kesimpulan dari hasil penelitian yang telah dilakukan serta saran – saran yang dapat digunakan untuk pengembangan sistem di masa mendatang

## BAB II

### TINJAUAN PUSTAKA

#### A. Kajian Teori

##### 1. Penyakit Bawang Merah

Bawang merah adalah salah satu jenis umbi-umbian yang dikonsumsi oleh masyarakat Indonesia. Sebagai salah satu dari tiga anggota genus *Allium*, bawang merah sangat digemari dan memiliki nilai ekonomi yang tinggi. Dalam budidayanya, bawang merah sering mengalami infeksi virus dan jamur karena rentan terhadap penyakit. Jika masalah ini tidak terdeteksi sejak awal, hal itu dapat mengakibatkan penurunan hasil panen bagi para petani (Nur dkk., 2022) .

##### a. Penyakit Trotol



**Gambar 2. 1** Penyakit Trotol Pada Bawang Merah  
(Sumber : Badan Litbang Pertanian)

Penyakit trotol atau bercak ungu merupakan salah satu penyakit pada tanaman bawang merah yang disebabkan oleh jamur *Alternaria porri* (Ell.Cif.). Adapun gejala yang ditimbulkan yaitu infeksi awal pada daun

menimbulkan bercak berukuran kecil, melekok ke dalam, berwarna putih dengan pusat yang berwarna ungu (kelabu) (Joeniarti dkk., 2024).

Upaya pencegahan dapat dilakukan dengan menerapkan rotasi tanaman menggunakan tanaman bukan inang, pemilihan benih sehat, serta membuat sistem drainase yang baik guna menghindari genang air di lahan (Nurfarm, 2021). Selain itu, penggunaan pupuk organik yang diperkaya agens hayati seperti *Trichoderma sp.* sangat dianjurkan karena efektif menekan pertumbuhan jamur tanpa menimbulkan dampak buruk bagi lingkungan maupun tanaman (Deden & Wijaya, 2023). Apabila tanaman yang terserang berat, langkah penanganan fisik yang dapat diambil meliputi pencabutan dan pemusnahan tanaman, serta sanitasi lahan dengan membakar sisa tanaman sakit (Distan Buleleng ,2021). Jika serangan telah melampaui ambang ekonomi, pengendalian kimiawi dapat dilakukan menggunakan fungisida berbahan aktif pilihan sesuai dosis anjuran yang diberikan pada awal usia tanaman atau saat gejala penyakit mulai terdeteksi (Susandi dkk., t.t.).

b. Penyakit Moler



**Gambar 2. 2** Penyakit Moler Pada Bawang Merah  
(Sumber : Dinas Pertanian, 2021 (distan.buleleng.go.id))

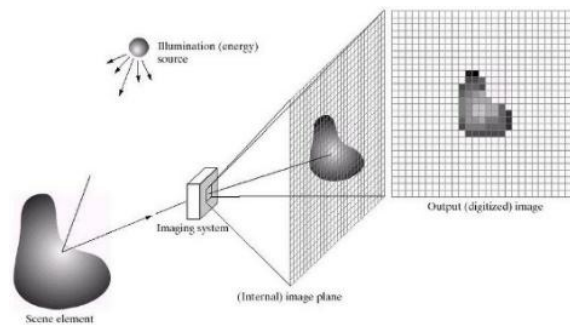
Penyakit moler pada tanaman bawang merah merupakan gangguan serius yang disebabkan oleh cendawan *Fusarium oxysporum* f.sp. *cepae*. Penyakit ini menyerang bagian umbi dan akar, dengan gejala khas adanya perubahan warna menjadi kecoklatan atau kehitaman serta pembusukan yang membuat umbi menjadi lembek dan mudah hancur.

Penularan umumnya terjadi melalui tanah tercemar spora jamur atau penggunaan bibit terinfeksi, yang diperparah oleh kondisi suhu hangat dan kelembapan tinggi. Oleh karena itu, diperlukan pencegahan dan pengendalian sanitasi lahan, serta aplikasi fungsida yang tepat guna menekan kerugian hasil panen.

c. Daun Sehat dan Bukan Bawang Merah

Selain kedua penyakit tersebut, sistem klasifikasi dalam penelitian ini juga mengenali kelas daun sehat (tanaman bawang merah tanpa gejala penyakit) serta kelas bukan bawang merah, yaitu citra yang bukan merupakan objek daun maupun umbi bawang merah. Penambahan kedua kelas ini bertujuan agar model tidak hanya mampu membedakan jenis penyakit, tetapi juga dapat mengenali kondisi tanaman yang sehat serta menolak citra masukan yang tidak relevan dengan objek penelitian, sehingga hasil klasifikasi yang diberikan kepada pengguna menjadi lebih akurat dan dapat dipertanggung jawabkan.

## 2. Citra Digital



**Gambar 2. 3** Alur Pengolahan Citra  
(Sumber : <https://pemrogramanmatlab.com>)

Gambaran atau representasi dari sebuah objek dikenal sebagai citra. Bentuknya beragam, mulai dari foto optik dan sinyal video *analog* seperti tampilan pada televisi hingga data format digital yang dapat disimpan dalam media penyimpanan. Secara khusus, citra digital tersusun atas matriks dua dimensi berisikan elemen terkecil bernama piksel yang menempati baris dan kolom. Nilai pada piksel inilah yang menentukan warna pada monitor. Kategori citra digital umumnya terbagi menjadi tiga jenis: citra *binary* (monokrom), citra skala keabuan (*grayscale*), dan citra warna (*true color*). Citra warna menggunakan tiga nilai piksel, yaitu merah, hijau dan biru, untuk membentuk berbagai macam warna yang dapat ditampilkan pada layar monitor (Allam & Wibowo, 2023).

Citra warna asli (*true color*) dibentuk oleh kombinasi tiga saluran warna dasar pada tiap pikselnya, yaitu merah (*Red*), hijau (*Green*), dan biru (*Blue*). Masing-masing komponen warna ini menggunakan kedalaman 8 bit, yang berarti nilainya berkisar antara 0 sampai 255. Dengan menggabungkan ketiga nilai R, G, dan B tersebut, sebuah piksel mampu menghasilkan hingga



16.581.375 variasi warna ( $255^3$ ). Secara keseluruhan, setiap piksel pada citra jenis ini membutuhkan alokasi memori sebesar 24 bit (hasil penjumlahan 8-bit dari ketiga elemen warnanya). Beberapa operasi augmentasi citra yang umum dilakukan meliputi:

- a. Cropping merupakan proses memangkas bagian tertentu pada gambar.
- b. Flipping merupakan tindakan mengubah arah gambar baik secara mendatar (*Horizontal*), tegak (*vertikal*), ataupun keduanya.
- c. Rotasi merupakan proses memutar posisi gambar sesuai sudut derajat tertentu mengelilingi titik porosnya, baik mengikuti arah jarum jam maupun sebaliknya.
- d. Resizing merupakan prosedur untuk mengubah dimensi atau resolusi citra agar ukurannya sesuai dengan kebutuhan yang diinginkan.

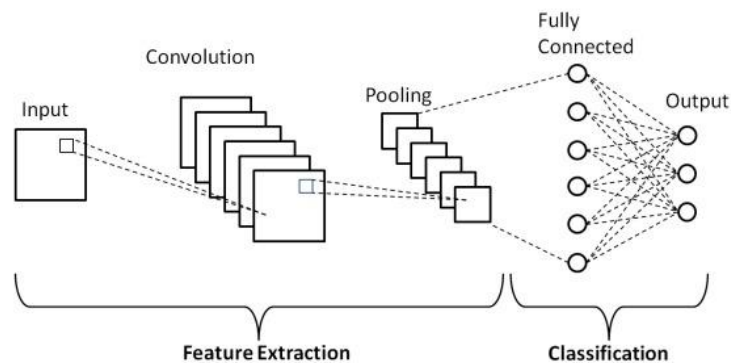
### 3. *Deep Learning*

*Deep learning* merupakan salah satu cabang dari kecerdasan buatan (*artificial intelligence*) yang menggunakan jaringan saraf tiruan berlapis-lapis (*multi-layer neural networks*) untuk mempelajari pola dan fitur dari data dalam jumlah besar secara otomatis. Teknologi ini telah berkembang pesat, khususnya untuk deteksi serta klasifikasi penyakit tanaman berbasis citra daun. Menurut Das (2024), *deep learning* mampu mengotomatisasi proses identifikasi penyakit tanaman yang sebelumnya sangat bergantung pada keahlian visual manusia dan proses manual yang memakan waktu.

Dalam konteks pertanian modern, penerapan *deep learning* telah menjadi pendekatan utama untuk mengatasi permasalahan klasifikasi citra

pertanian. Teknologi ini dirancang khusus untuk mengenali pola visual yang kompleks secara otomatis serta efektif dalam membedakan kondisi tanaman yang sehat dari tanaman yang terinfeksi oleh pantogen tertentu. Studi oleh Ali et al. (2024) menunjukkan bahwa pemanfaatan arsitektur *deep learning* mampu meningkatkan akurasi klasifikasi penyakit tanaman secara signifikan, bahkan pada kondisi data pelatihan yang tidak seimbang (*imbalanced dataset*). Keunggulan utama paradigma ini terletak pada kemampuannya untuk melakukan ekstraksi fitur secara mandiri tanpa memerlukan intervensi rekayasa fitur manual dari manusia.

#### 4. *Convolutional Neural Network* (CNN)



**Gambar 2. 4** Alur *Convolutional Neural Network*

(Sumber : [https://www.researchgate.net/figure/Schematic-diagram-of-a-basic-convolutional-neural-network-CNN-architecture-26\\_fig1\\_336805909](https://www.researchgate.net/figure/Schematic-diagram-of-a-basic-convolutional-neural-network-CNN-architecture-26_fig1_336805909))

*Convolutional Neural Network* (CNN) merupakan salah satu arsitektur *deep learning* yang paling populer untuk pengolahan data berbasis grid, seperti citra digital. Arsitektur CNN terinspirasi dari cara kerja korteks visual pada makhluk hidup, di mana neuron-neuron hanya merespons pada area tertentu dari bidang pandang yang disebut *receptive field*.

Secara umum, arsitektur CNN terbagi menjadi dua tahapan besar, yaitu *Feature Extraction Layer* (lapisan ekstraksi fitur) yang terdiri dari operasi *convolutional layer*, ReLU, dan *pooling layer*, serta *Classification Layer* (lapisan klasifikasi) yang terdiri dari proses *flattening*, dan *fully connected layer*, yang disusun secara berurutan untuk mengekstraksi fitur secara hierarkis mulai dari fitur rendah (tepi dan warna) hingga fitur tingkat tinggi (bentuk dan objek). Adapun penjelasan lebih rinci mengenai cara kerja CNN dapat diuraikan yaitu sebagai berikut:

a. *Input Layer*

*Input layer* pada *Convolutional Neural Network* (CNN) bertugas menerima data citra yang diproses oleh mode. Sebagai contoh, sebuah citra berformat RGB dengan dimensi 224 x 224 pixel akan ditransformasikan ke dalam matriks berukuran 224 x 224 x 3. Disini, angka 3 menandakan tiga saluran warna utama yaitu *Red*, *Green*, dan *Blue*.

b. *Convolutional Layer*

Lapisan konvolusi (*convolutional layer*) bertugas memproses citra masukan yang telah dipecah menjadi tiga saluran warna dasar (merah, hijau, dan biru), sehingga membentuk matriks multidimensi berukuran  $n \times n \times 3$ . Pada lapisan ini, terdapat susunan neuron berupa matriks filter (*kernel*) dengan dimensi spesifik yang akan dikomputasikan dengan ketiga kanal RGB masukan. Hasil perhitungan matematika ini kemudian melahirkan matriks fitur baru yang disebut sebagai *output layer*. Bergantung pada *hyperparameter* yang dikonfigurasi, operasi konvolusi ini dapat berulang

hingga puluhan, ratusan, bahkan ribuan kali. Adapun rumus operasi konvolusi ditunjukkan pada persamaan (2.1) :

$$S(i, j) = (K * I)(i, j) + b = \sum_{m=1}^M \sum_{n=1}^N (I(i - m, j - n) * K)(m, n) + b \dots (2.1)$$

Adapun rumus *convolutional layer* adalah persamaan (2.2)

$$N_{\text{out}} = \frac{N_{\text{in}} - F + 2P}{S} + 1 \dots (2.2)$$

c. *Rectified Linear Units (ReLU) layer*

Fungsi aktivasi digunakan untuk memproses keluaran yang dihasilkan oleh *convolutional layer*. Dalam arsitektur CNN fungsi aktivasi *Rectified Linear Units (ReLU)* mengubah output nilai negatif menjadi nilai 0 (Setiawan, 2020). Adapun rumus fungsi aktivasi ReLU sebagai berikut.

$$f(x) = \max(0, x) \dots (2.3)$$

d. *Pooling Layer*

*Pooling layer* berperan dalam mereduksi dimensi matriks *feature map*, karena tidak semua nilai piksel relevan. Lapisan ini berfungsi untuk mengefisiensikan durasi komputasi dengan tetap mempertahankan piksel-piksel kunci yang memuat informasi penting. Terdapat dua jenis *pooling layer*, yaitu *max pooling* dan *average pooling*, namun *pooling* yang paling sering digunakan di CNN adalah *max pooling* dengan mengubah keluaran lapisan konvolusi menjadi beberapa *grid* kecil dan mengambil nilai maksimum dari setiap *grid*.

e. *Flattening Layer*

*Flattening* merupakan tahapan yang lazim dilakukan sebelum masuk *Fully Connected Layer*. *Feature map* hasil dari tahap *feature extraction* masih berbentuk tensor multidimensi, sehingga perlu dilakukan proses *flattening* untuk mengubah *reshape feature map* menjadi vektor satu dimensi agar dapat digunakan sebagai input pada tahap klasifikasi. Proses ini tidak mengubah nilai yang ada pada *feature map*, melainkan hanya mengubah susunannya, sehingga informasi yang telah dipelajari pada tahap sebelumnya tetap terjaga dalam format yang sesuai untuk diproses oleh lapisan *fully connected* (Juliansyah & Laksito, 2021).

Meskipun *flattening* umum digunakan, pendekatan ini memiliki kelemahan yaitu menghasilkan vektor berdimensi sangat besar ketika *feature map* memiliki banyak *channel*, sehingga jumlah parameter pada lapisan *fully connected* berikutnya menjadi sangat besar dan berpotensi menyebabkan *overfitting*, terutama pada dataset dengan jumlah data yang terbatas. Sebagai alternatif, penelitian ini menggunakan *Global Average Pooling* (GAP) untuk menggantikan fungsi *flattening*. GAP bekerja dengan menghitung nilai rata-rata dari setiap *feature map* (*channel*) hasil ekstraksi fitur pada lapisan konvolusi terakhir, sehingga setiap *channel* diringkas menjadi satu nilai skalar tunggal. Jika *feature map* berukuran  $H \times W \times C$ , maka keluaran GAP berukuran  $1 \times 1 \times C$ , dengan rumus sebagai berikut:

$$GAP(x)_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W x_{i,j,c} \dots\dots\dots (2.4)$$

Penggunaan GAP dipilih karena mampu mengurangi jumlah parameter secara signifikan dibandingkan flattening konvensional, tanpa perlu lapisan tambahan untuk regularisasi, sehingga menekan risiko overfitting sekaligus mempercepat proses komputasi. Selain itu, GAP juga lebih tahan terhadap perubahan posisi spasial fitur pada citra input, karena hasil akhirnya merupakan representasi rata-rata dari keseluruhan feature map, bukan susunan nilai piksel yang bergantung pada posisi

f. *Fully Connected Layer*

*Fully Connected Layer* menerima output dari lapisan Global Average Pooling untuk mengekstrak fitur yang paling relevan dengan kelas target. Salah satu fungsi dasarnya adalah mentransformasikan seluruh nodus yang mulanya tersusun dalam matriks dua dimensi menjadi bentuk vektor tunggal, yang umumnya dikenal sebagai proses *flattening* (Asrianda dkk., 2021). Selanjutnya, *Fully Connected Layer* mengolah data tersebut untuk kebutuhan klasifikasi, di mana luaran yang dihasilkan berupa nilai probabilitas terhadap tiap kategori, khususnya jika dikombinasikan dengan fungsi aktivasi *Softmax* (Allam & Wibowo, 2023).

Pada lapisan *output layer* dalam tahap *fully connected* atau klasifikasi yang melibatkan lebih dari 2 kelas, fungsi aktivasi *softmax* berperan penting. Fungsi ini menghitung peluang setiap kelas target secara relatif terhadap seluruh kemungkinan kelas yang ada, sehingga memudahkan penentuan kategori yang paling akurat untuk data citra masukan. Keunggulan utama dari *Softmax* ialah menghasilkan nilai probabilitas dalam rentang 0 sampai 1,

dengan total keseluruhan probabilitas bernilai. Kelas dengan nilai probabilitas tertinggi kemudian dipilih sebagai hasil prediksi akhir model. Dengan demikian, lapisan *Fully Connected* berperan mengekstraksi kombinasi fitur yang paling relevan untuk membedakan tiap kelas, sedangkan Softmax berperan menerjemahkan hasil ekstraksi tersebut ke dalam bentuk probabilitas yang mudah diinterpretasikan, sehingga keduanya lazim digunakan sebagai satu kesatuan pada lapisan keluaran arsitektur CNN untuk tugas klasifikasi multi kelas. Berikut rumus *softmax*:

$$\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \dots\dots\dots (2.5)$$

## 5. *Confussion Matriks*

*Confusion matrix* merupakan sebuah tabel yang digunakan untuk mengevaluasi kinerja model klasifikasi dengan cara membandingkan hasil prediksi model terhadap label sebenarnya (*ground truth*) pada data uji. Melalui *confusion matrix*, dapat diketahui secara rinci jumlah data yang diklasifikasikan dengan benar maupun yang keliru untuk setiap kelas, sehingga evaluasi performa model menjadi lebih informatif dibandingkan hanya melihat nilai akurasi secara keseluruhan.

Pada kasus klasifikasi dengan lebih dari dua kelas, seperti pada penelitian ini yang mengklasifikasikan citra ke dalam empat kelas (Trotol, Moler, Daun Sehat, dan Bukan Bawang Merah), *confusion matrix* disusun dalam bentuk matriks berukuran  $n \times n$ , dengan  $n$  merupakan jumlah kelas. Baris pada matriks merepresentasikan kelas aktual, sedangkan kolom merepresentasikan kelas hasil prediksi model. Nilai pada diagonal utama matriks menunjukkan jumlah prediksi

yang benar (*True Positive*/TP untuk masing-masing kelas), sementara nilai di luar diagonal menunjukkan jumlah kesalahan klasifikasi, yaitu *False Positive* (FP) dan *False Negative* (FN).

Berdasarkan nilai TP, FP, FN, dan *True Negative* (TN) pada *confusion matrix*, dapat dihitung beberapa metrik evaluasi untuk menilai kualitas model klasifikasi, yaitu:

- a. *Accuracy* menunjukkan persentase jumlah prediksi yang benar dibandingkan dengan keseluruhan data yang diuji, dirumuskan sebagai berikut.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \dots\dots\dots (2.5)$$

Keterangan:

- TP adalah *True Positives* (Prediksi benar untuk kelas positif).
- TN adalah *True Negatives* (Prediksi benar untuk kelas negatif).
- FP adalah *False Potives* (Prediksi salah untuk kelas positif).
- FN adalah *False Negatives* (Prediksi salah untuk kelas negatif).

- b. *Precision* menunjukkan proporsi prediksi positif suatu kelas yang benar-benar tepat, dirumuskan sebagai berikut.

$$Precision = \frac{TP}{TP+FP} \dots\dots\dots (2.6)$$

- c. *Recall* menunjukkan kemampuan model dalam mengenali seluruh data positif yang sebenarnya pada suatu kelas, dirumuskan sebagai berikut.

$$Recall = \frac{TP}{TP+FN} \dots\dots\dots (2.7)$$

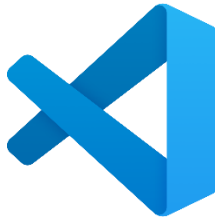
- d. *F1-Score* merupakan nilai rata-rata harmonik antara *precision* dan *recall*, yang digunakan untuk mendapatkan gambaran yang seimbang mengenai



performa model, terutama pada kondisi distribusi data antar kelas yang tidak seimbang, dirumuskan sebagai berikut.

$$F1\ Score = 2 \times \frac{Recall \times Precision}{Recall + Precision} \dots\dots\dots (2.8)$$

## 6. *Visual Studio Code*



**Gambar 2. 5** *Visual Studio Code*  
(Sumber: <https://code.visualstudio.com>)

Visual Studio Code (VS Code) merupakan editor kode sumber terbuka dan gratis yang dikembangkan oleh Microsoft untuk berbagai sistem operasi, termasuk Windows, Linux, dan macOS. Menurut Murani dkk. (2025), secara teoretis VS Code dikategorikan sebagai Integrated Development Environment (IDE) yang ringan namun kuat, dirancang untuk mendukung berbagai bahasa pemrograman melalui ekstensi yang dapat ditambahkan sesuai kebutuhan. Arsitektur VS Code menyediakan berbagai fitur yang membantu programmer dalam menulis, men-debug, dan mengelola kode, seperti IntelliSense (saran kode cerdas berdasarkan konteks), debugging terintegrasi, integrasi Git untuk kontrol versi, serta antarmuka yang bersih dan intuitif.

Keunggulan utama VS Code dalam konteks pengembangan perangkat lunak adalah kemampuannya yang fleksibel dan efisien untuk berbagai tingkat keahlian, mulai dari pemula hingga profesional. Menurut Murani dkk. (2025), penelitian mereka menunjukkan bahwa VS Code sangat direkomendasikan bagi

pemula dalam pembelajaran pemrograman karena 90% responden mahasiswa informatika menyatakan bahwa VS Code mudah digunakan, memiliki fitur yang lengkap, dan efektif untuk meningkatkan produktivitas coding. Fitur-fitur seperti auto-save, kustomisasi tema, dukungan ekstensi yang luas, dan terminal terintegrasi menjadikan VS Code alat yang unggul dibandingkan editor kode lainnya dalam mendukung proses belajar dan pengembangan perangkat lunak.

## 7. *Python*



**Gambar 2. 6** *Python*

Python merupakan bahasa pemrograman utama dalam pengembangan sistem *deep learning* modern, termasuk arsitektur *Convolutional Neural Network* (CNN). Python dipilih karena sintaksnya sederhana, komunitasnya luas, serta ekosistem pustaka yang sangat kaya dan mendukung seluruh tahapan *pipeline deep learning*, mulai dari pra-pemrosesan data hingga *deployment* model (Singh dkk., 2025).

Dalam implementasi sistem klasifikasi penyakit tanaman berbasis *Convolutional Neural Network* (CNN), bahasa pemrograman Python menjadi fondasi utama karena menyediakan ekosistem pustaka yang sangat lengkap dan terintegrasi untuk seluruh tahapan pemrosesan data dan pengembangan model.

a. *NumPy*

*NumPy* merupakan pustaka dasar yang digunakan untuk manipulasi data numerik dan pengelolaan *dataset*. *NumPy* memungkinkan transformasi citra digital menjadi *array* numerik multidimensi, sehingga data gambar dapat diproses secara efisien oleh model *deep learning*.

b. *Tensorflow* dan Keras

Inti pengembangan model, *TensorFlow* dan *Keras* menjadi framework utama yang digunakan untuk membangun, melatih, dan menguji arsitektur *Convolutional Neural Network* (CNN). *Keras* menyediakan API modular yang memudahkan perancangan layer konvolusi (CNN) untuk ekstraksi fitur citra, serta integrasi dengan layer CNN untuk pemrosesan urutan fitur secara efisien. Kombinasi ini memungkinkan pengembangan model yang mampu menangkap pola spasial dan temporal secara bersamaan, sehingga meningkatkan akurasi klasifikasi penyakit tanaman (Kumar et al., 2023).

c. *Pillow*

*Pillow* (*Python Imaging Library*) merupakan salah satu pustaka Python yang digunakan untuk mengolah dan memanipulasi citra digital. Pustaka ini mendukung berbagai format gambar, seperti JPEG, PNG, dan BMP, serta menyediakan fungsi untuk membuka, mengubah, dan menyimpan citra (Schmidt dkk., 2026). Pada penelitian ini, *Pillow* digunakan dalam tahap pra-pemrosesan citra daun bawang merah sebelum citra diproses oleh model CNN. Beberapa proses yang dilakukan meliputi *resizing* untuk menyesuaikan

ukuran citra dengan kebutuhan model, *cropping* untuk mengambil bagian daun yang menjadi objek utama, serta konversi mode warna, misalnya dari RGBA menjadi RGB. Selain itu, Pillow juga dapat digunakan untuk membantu penyesuaian nilai piksel sebelum citra diubah ke bentuk yang dapat diterima oleh model. Tahap pra-pemrosesan ini diperlukan agar citra yang digunakan sebagai masukan memiliki ukuran dan format yang sesuai, sehingga proses klasifikasi oleh model *deep learning* dapat berjalan dengan baik.

d. *Scikit-learn*

Scikit-learn merupakan pustaka *machine learning* berbasis Python yang menyediakan berbagai algoritma untuk klasifikasi, regresi, klustering, serta beragam fungsi pendukung dalam pra-pemrosesan data dan evaluasi model (Koukaras & Tjortjis, 2025). Pada penelitian ini, Scikit-learn tidak digunakan untuk membangun model klasifikasi utama karena proses pelatihan dilakukan menggunakan TensorFlow dan Keras. Sebaliknya, pustaka ini dimanfaatkan untuk mengevaluasi kinerja model melalui perhitungan metrik seperti akurasi, presisi, *recall*, dan *F1-score*, serta penyusunan *confusion matrix* untuk melihat distribusi prediksi yang benar maupun salah pada setiap kelas. Selain itu, Scikit-learn digunakan dalam proses pembagian data menjadi data latih dan data uji (*train-test split*), sehingga pengujian model dapat dilakukan secara lebih objektif.

e. *Matplotlib*

Matplotlib adalah pustaka visualisasi data Python yang digunakan untuk menghasilkan grafik dan plot dua dimensi berkualitas publikasi, termasuk line chart, bar chart, histogram, dan heatmap (Idkowiak dkk., 2025). Dalam penelitian ini, Matplotlib dimanfaatkan untuk memvisualisasikan berbagai aspek hasil eksperimen, seperti kurva pelatihan (*training curve*) yang menampilkan perubahan nilai *loss* dan akurasi selama proses pelatihan model, matriks konfusi yang menggambarkan performa klasifikasi untuk setiap kelas, serta grafik distribusi kelas dalam dataset untuk mengidentifikasi ketidakseimbangan data. Visualisasi yang dihasilkan oleh Matplotlib tidak hanya memudahkan interpretasi hasil analisis oleh peneliti, tetapi juga mendukung penyajian data yang jelas dan informatif dalam laporan penelitian atau artikel ilmiah. Dengan kemampuan kustomisasi yang tinggi, Matplotlib memungkinkan pembuatan grafik yang sesuai dengan standar akademik dan persyaratan publikasi jurnal.

f. *SQLite*

SQLite merupakan sistem manajemen basis data relasional ringan yang banyak digunakan dalam aplikasi mobile dan *embedded systems* karena efisiensinya dalam menyimpan dan mengelola data lokal. Dalam konteks pertanian pintar, SQLite sering digunakan untuk menyimpan dataset citra tanaman, hasil prediksi model deep learning, serta metadata terkait kondisi pertumbuhan tanaman. Misalnya, dalam penelitian oleh (Qin dkk., 2025) SQLite digunakan untuk menyimpan data hasil klasifikasi penyakit pada

tanaman selada yang diperoleh melalui model deep learning berbasis CNN. Penggunaan SQLite memungkinkan integrasi yang mulus antara *backend server* dan aplikasi mobile, serta mendukung pembaruan data secara real-time melalui antarmuka pengguna yang sederhana dan responsif.

Dengan dukungan pustaka-pustaka tersebut, Python tidak hanya mempercepat proses pengembangan dan eksperimen model *deep learning*, tetapi juga memastikan reproduisibilitas dan kemudahan integrasi sistem ke dalam aplikasi nyata, seperti sistem deteksi penyakit tanaman berbasis web atau mobile.

#### 8. HTML (*HyperText Markup Language*)



**Gambar 2. 7** HTML (*Hypertext Markup Language*)  
(Sumber: <https://icons8.com/icons/set/html-logo>)

HTML, singkatan dari Hypertext Markup Language, adalah bahasa standar web yang diatur penggunaannya oleh W3C (*World Wide Web Consortium*). HTML terdiri dari kumpulan tag yang digunakan untuk menyusun elemen-elemen dalam sebuah website. Perannya adalah sebagai penyusun struktur halaman web, memungkinkan setiap elemen ditempatkan sesuai dengan tata letak (*layout*) yang diinginkan (Permata Sari, 2020). HTML dirancang untuk menampilkan berbagai komponen *web*, termasuk teks, gambar, tautan, tabel, formulir, dan elemen multimedia lainnya. Kode HTML ditulis dalam bentuk

dokumen teks biasa dan kemudian diinterpretasikan oleh 18 peramban *web* (*browser*), yang akan menghasilkan halaman *web* interaktif dan dapat diakses pengguna. Fungsi utama HTML digunakan untuk mengelola dan menyusun data serta informasi agar dapat diakses dan ditampilkan di internet melalui peramban web (*browser*). HTML juga memungkinkan penghubungan antar halaman web melalui tautan (*hyperlink*). HTML terdiri atas tiga bagian utama:

- a. Tag yaitu elemen yang digunakan untuk menandai komponen dalam dokumen. Misalnya, `<p>` untuk paragraph atau `<img>` untuk gambar.
- b. Atribut adalah properti tambahan pada tag untuk memberikan informasi lebih spesifik, seperti `src` pada tag `<img>` untuk menentukan sumber gambar.
- c. Konten yaitu data atau informasi yang berada di antara tag pembuka dan penutup.

## 9. CSS

Cascading Style Sheets (CSS) adalah sebuah bahasa desain web yang digunakan untuk mengatur tampilan serta format halaman website. CSS memiliki peran penting dalam memisahkan antara konten dan tampilan pada halaman web, sehingga memungkinkan pengelolaan dan modifikasi desain tanpa harus mengubah struktur utama dari konten HTML (Yuliana, D., & Kresna, I. A. 2022). Dengan CSS, pengembang dapat mengatur elemen visual seperti warna, font, tata letak, dan latar belakang secara lebih mudah dan konsisten di seluruh halaman sebuah situs. Hal ini memberikan fleksibilitas tinggi dalam pengembangan tampilan web, memudahkan pembaruan desain,

serta menghasilkan user interface yang lebih menarik dan responsif di berbagai perangkat.

Fungsi CSS sangat menonjol dalam penelitian pengembangan web modern, terutama untuk meningkatkan estetika, keterbacaan, dan pengalaman pengguna pada aplikasi berbasis web. Penerapan CSS lazim digunakan bersama HTML dan *JavaScript* untuk membangun tampilan halaman yang interaktif, seragam, serta meningkatkan aksesibilitas dan navigasi situs. Selain itu, CSS juga mendukung penggunaan *framework* seperti *Bootstrap* atau *Tailwind CSS*, yang mempercepat proses pembuatan desain *website* responsif. Dengan demikian, penggunaan CSS dalam penelitian pengembangan web terbukti meningkatkan kelayakan hasil tata letak dan visual, serta mendukung tujuan penelitian dalam aspek implementasi teknologi modern.

## 10. JavaScript

Java Script adalah bahasa pemrograman yang berjalan di sisi klien (*client-side*) pada peramban web dan memungkinkan halaman web menjadi interaktif serta dinamis. Berbeda dengan HTML yang menyusun struktur dan CSS yang mengatur tampilan, JavaScript berperan mengendalikan perilaku halaman, seperti merespons klik pengguna, memvalidasi input formulir, serta memperbaharui tampilan tanpa perlu memuat ulang seluruh halaman (Permata Sari, 2020).

Dalam penelitian ini, JavaScript digunakan untuk membangun antarmuka sistem klasifikasi penyakit bawang merah, mulai dari pengambilan gambar melalui kamera perangkat, penampilan hasil prediksi model secara *real-*



*time*, hingga visualisasi tahapan *Convolutinal Neural Network* (CNN) seperti *convolution*, *ReLU*, *pooling*, *flatten*, *fully connected*, dan *output* kepada pengguna. JavaScript juga berperan menghubungkan antarmuka pengguna dengan layanan *backend* melalui permintaan *asynchronous request*, sehingga data yang diproses oleh model dapat ditampilkan kepada pengguna secara cepat dan responsif tanpa memuat ulang halaman.

## 11. FastAPI

FastAPI merupakan framework web berbasis Python yang digunakan untuk membangun *Application Programming Interface* (API) secara cepat dan efisien. Framework ini memanfaatkan Starlette sebagai dasar pengembangan aplikasi web asinkronus dengan standar ASGI (*Asynchronous Server Gateway Interface*), serta Pydantic untuk melakukan validasi data berdasarkan *type hints* pada Python. Kombinasi keduanya membantu memastikan data masukan dan keluaran sesuai dengan struktur yang telah ditentukan (Bednarz & Miłosz, 2025). FastAPI juga mendukung pemrograman asinkronus melalui *async* dan *await*, yang memungkinkan server menangani beberapa permintaan secara bersamaan, terutama pada proses I/O. Hal ini menjadikannya sesuai untuk sistem klasifikasi citra berbasis *deep learning* yang melibatkan interaksi antara pengguna, server, dan model dalam proses prediksi (Nematzadeh dkk., 2026). Selain itu, FastAPI menyediakan fitur dokumentasi API otomatis berdasarkan standar OpenAPI. Dokumentasi ini dapat diakses melalui Swagger UI maupun ReDoc, sehingga setiap *endpoint* dapat diperiksa dan diuji dengan lebih mudah. Fitur tersebut mempermudah proses pengembangan karena struktur input,

output, dan fungsi setiap *endpoint* dapat dipahami tanpa perlu dokumentasi tambahan. Dengan demikian, FastAPI menjadi salah satu pilihan yang efektif dalam pengembangan API untuk aplikasi *machine learning* dan *artificial intelligence*.

Pada penelitian ini, FastAPI digunakan sebagai backend yang menghubungkan antarmuka web dengan model *Convolutional Neural Network* (CNN) yang telah dilatih menggunakan TensorFlow dan Keras. *Endpoint* */predict* digunakan untuk menerima citra daun bawang merah dalam format JPEG atau PNG, baik dari unggahan pengguna maupun kamera sistem. Citra yang diterima kemudian dipra-proses melalui tahap *resizing*, normalisasi piksel, dan konversi ke bentuk tensor sebelum dimasukkan ke model CNN. Model kemudian menghasilkan nilai probabilitas untuk setiap kelas, dan kelas dengan nilai tertinggi dipilih sebagai hasil prediksi. Hasil tersebut dikirim kembali dalam format JSON yang berisi label kelas dan tingkat kepercayaan, serta dapat disertai informasi tambahan jika diperlukan.

#### 10. *Unified Modeling Language* (UML)



**Gambar 2. 8** *Unified Modeling Language*

*Unified Modeling Language* (UML) adalah bahasa pemodelan yang digunakan dalam pengembangan perangkat lunak untuk memvisualisasi,

menspesifikasikan, membangun dan mendokumentasikan sistem perangkat lunak berorientasi objek (Destriana et al., 2021).

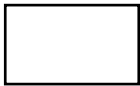
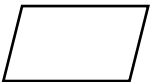
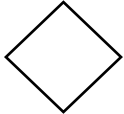
UML menyediakan notasi yang digunakan untuk menggambarkan berbagai elemen dan interaksi dalam sistem perangkat lunak. Notasi UML sangat penting karena memungkinkan pengembang dan pengguna akhir untuk berkomunikasi secara jelas dan konsisten mengenai desain dan struktur sistem.

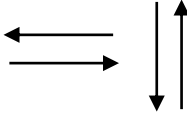
#### a. Flowchart

Menurut Smrti, dkk (2023) *Flowchart* adalah diagram yang digunakan untuk menggambarkan tahapan penyelesaian suatu masalah yang ditulis atau dilambangkan dengan simbol-simbol tertentu. *Flowchart* digunakan untuk mendokumentasikan, menganalisis, merencanakan, dan memahami proses-proses kompleks dengan cara yang mudah dipahami. Diagram ini tidak hanya berguna untuk komunikasi, tetapi juga sebagai alat yang efektif untuk menggambarkan alur prosedur atau sistem.

Berikut adalah simbol-simbol yang umum atau sering dijumpai dalam penggunaan proses pembuatan *flowchart*:

**Tabel 2. 1** Simbol *Flowchart* umum digunakan

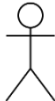
| Simbol                                                                              | Nama                | Keterangan                                                             |
|-------------------------------------------------------------------------------------|---------------------|------------------------------------------------------------------------|
|  | <i>Process</i>      | Digunakan untuk menyatakan sebuah proses yang dilakukan oleh komputer. |
|  | <i>Input/Output</i> | Digunakan untuk menyatakan sebuah masukan atau keluaran.               |
|  | <i>Decision</i>     | Digunakan untuk menyatakan sebuah kondisi yang menghasilkan dua hasil. |

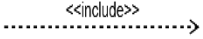
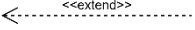
|                                                                                   |                          |                                                                                       |
|-----------------------------------------------------------------------------------|--------------------------|---------------------------------------------------------------------------------------|
|  | <i>Terminator</i>        | Digunakan untuk menandai awal dan akhir <i>flowchart</i> .                            |
|  | <i>Flow</i>              | Digunakan untuk menghubungkan sebuah simbol dengan simbol yang lainnya.               |
|  | <i>Predefine Process</i> | Digunakan untuk menunjukkan suatu operasi yang rinciannya ditunjukkan di tempat lain. |
|  | <i>Offpage Connector</i> | Digunakan sebagai penghubung <i>flowchart</i> dalam lembar yang berbeda.              |
|  | <i>Connector</i>         | Digunakan sebagai penghubung <i>flowchart</i> dalam lembar yang sama.                 |

b. *Use Case*

*Use case* merupakan sarana untuk membantu merancang sistem dari sudut pandang pengguna akhir. Ini adalah metode untuk mengkomunikasikan perilaku sistem untuk pengguna dengan mengidentifikasi semua perilaku sistem yang terlihat dari luar. Diagram ini hanya merangkum beberapa hubungan antara kasus aktor dan sistem (Visual, 2022).

**Tabel 2. 2** Simbol *Use Case Diagram*

| <b>Simbol</b>                                                                       | <b>Nama</b>     | <b>Keterangan</b>                                                |
|-------------------------------------------------------------------------------------|-----------------|------------------------------------------------------------------|
|  | Aktor           | Merepresentasikan peran yang berinteraksi dengan <i>Use Case</i> |
|  | <i>Use Case</i> | Merupakan urutan interaksi antara aktor dengan sistem.           |

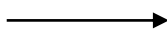
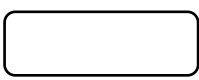
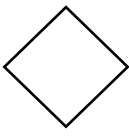
|                                                                                     |                                |                                                                                                                                                    |
|-------------------------------------------------------------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <i>Association</i>             | Penghubung antara aktor dan <i>use case</i> .                                                                                                      |
|    | <i>System</i>                  | Rancangan sistem yang akan dikembangkan atau dibuat.                                                                                               |
|    | <i>&lt;&lt;include&gt;&gt;</i> | Menunjukkan bahwa <i>use case</i> sepenuhnya adalah bagian operasi <i>use case</i> lainnya.                                                        |
|    | <i>&lt;&lt;extend&gt;&gt;</i>  | Menunjukkan bahwa jika kondisi atau syarat tertentu terpenuhi, suatu <i>use case</i> berfungsi sebagai pelengkap fungsi dari <i>use case</i> lain. |
|  | <i>Generalization</i>          | Hubungan dimana objek anak ( <i>descendent</i> ) berbagi perilaku dan struktur data dari objek yang ada di atasnya objek induk ( <i>ancestor</i> ) |

### c. Activity Diagram

Menurut Gedam & Meshram (2023) *activity* diagram menggambarkan alur dari satu aksi ke aksi lain dalam suatu sistem. Diagram ini menampilkan alur kegiatan yang terjadi dalam suatu sistem secara dinamis dengan memperlihatkan bagaimana sistem beroperasi.

**Tabel 2. 3** Simbol *Activity Diagram*

| <b>Simbol</b>                                                                       | <b>Nama</b>                          | <b>Keterangan</b>                                     |
|-------------------------------------------------------------------------------------|--------------------------------------|-------------------------------------------------------|
|  | <i>Initial Node</i><br>(Status Awal) | Sebuah diagram aktivitas memiliki sebuah status awal. |

|                                                                                   |                                     |                                                                                         |
|-----------------------------------------------------------------------------------|-------------------------------------|-----------------------------------------------------------------------------------------|
|  | <i>Final Node</i><br>(Status Akhir) | Status akhir yang dilakukan sistem, sebuah diagram aktivitas yang memiliki sebuah akhir |
|  | <i>Control Flow</i>                 | Menjelaskan aliran kontrol dari satu tindakan ke tindakan lain.                         |
|  | <i>Activity</i>                     | Aktivitas yang dilakukan sistem, aktivitas biasanya diawali dengan kata kerja.          |
|  | <i>Merge/Decision</i>               | Percabangan dimana ada pilihan aktivitas yang lebih dari satu.                          |
|  | <i>Fork Node/Join Node</i>          | Penggabungan dimana lebih dari satu aktivitas lalu digabungkan jadi satu.               |

d. *Sequence Diagram*

*Sequence diagram* digunakan untuk menggambarkan interaksi antara objek dalam suatu sistem berdasarkan urutan waktu. Diagram ini menunjukkan siklus hidup objek serta pesan yang dikirimkan atau diterima selama proses berlangsung. Phillip Mrzyglocki (2023) menuliskan bahwa diagram ini membantu pengembang untuk merinci urutan yang terjadi antar objek dalam sistem, serta mengilustrasikan hubungan antar pesan secara jelas dan terstruktur.

**Tabel 2. 4** Simbol *Sequence Diagram*

| Simbol | Nama | Keterangan |
|--------|------|------------|
|--------|------|------------|

|                                                                                   |                   |                                                                                                                                                    |
|-----------------------------------------------------------------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <i>Lifeline</i>   | Menyatakan kehidupan suatu objek                                                                                                                   |
|  | <i>Object</i>     | Menyatakan objek yang berinteraksi pesan                                                                                                           |
|  | <i>Activation</i> | Menyatakan objek dalam keadaan aktif dan berinteraksi, semua yang terhubung dengan waktu aktif ini adalah sebuah tahapan yang dilakukan didalamnya |

## 12. *Black-box Testing*



**Gambar 2. 9** *Black-box Testing*  
(Sumber: <https://thectoclub.com>)

Metode *Black-Box Testing* adalah metode pengujian perangkat lunak yang berfokus pada pengujian fungsionalitas suatu sistem tanpa memperhatikan struktur kode program di dalamnya. Menurut (Praniffa dkk., 2023) pengujian *black-box* dilakukan hanya berdasarkan spesifikasi kebutuhan sistem, dimana

penguji tidak memiliki akses terhadap kode sumber (*source code*) dan bertindak layaknya pengguna akhir yang menguji sistem melalui antarmuka yang tersedia.

*Black-box testing* bertujuan untuk memastikan bahwa keluaran (*output*) yang dihasilkan sistem telah sesuai dengan masukan (*input*). Namun, jika terjadi kesalahan dalam menjalankan prosedur, maka perlu perbaikan.

*Black-box testing* memiliki keterbatasan dan keunggulan dalam penerapannya. Salah satu keterbatasannya adalah pengujian yang tidak dapat dilakukan secara menyeluruh dikarenakan pengetahuan penguji terhadap struktur internal sistem. Namun metode ini memiliki kemampuan dalam mengidentifikasi ketidaksesuaian atau spesifikasi kebutuhan yang belum terpenuhi pada perangkat lunak (Praniffa dkk., 2023).

**Tabel 2. 5** Contoh Pengujian Black – Box Testing

| Test Faktor                                        | Hasil | Keterangan                                                                                                        |
|----------------------------------------------------|-------|-------------------------------------------------------------------------------------------------------------------|
| Pengguna mengunggah gambar berformat .jpg dan .png | ✓     | Berhasil menerima gambar, menampilkan pratinjau dan memproses gambar untuk dideteksi oleh model CNN               |
| Pengguna mengunggah file selain format gambar      | ✓     | Berhasil menolak file yang diunggah dan menampilkan pesan peringatan bahwa format file tidak didukung oleh sistem |



## B. Kajian Penelitian Terdahulu

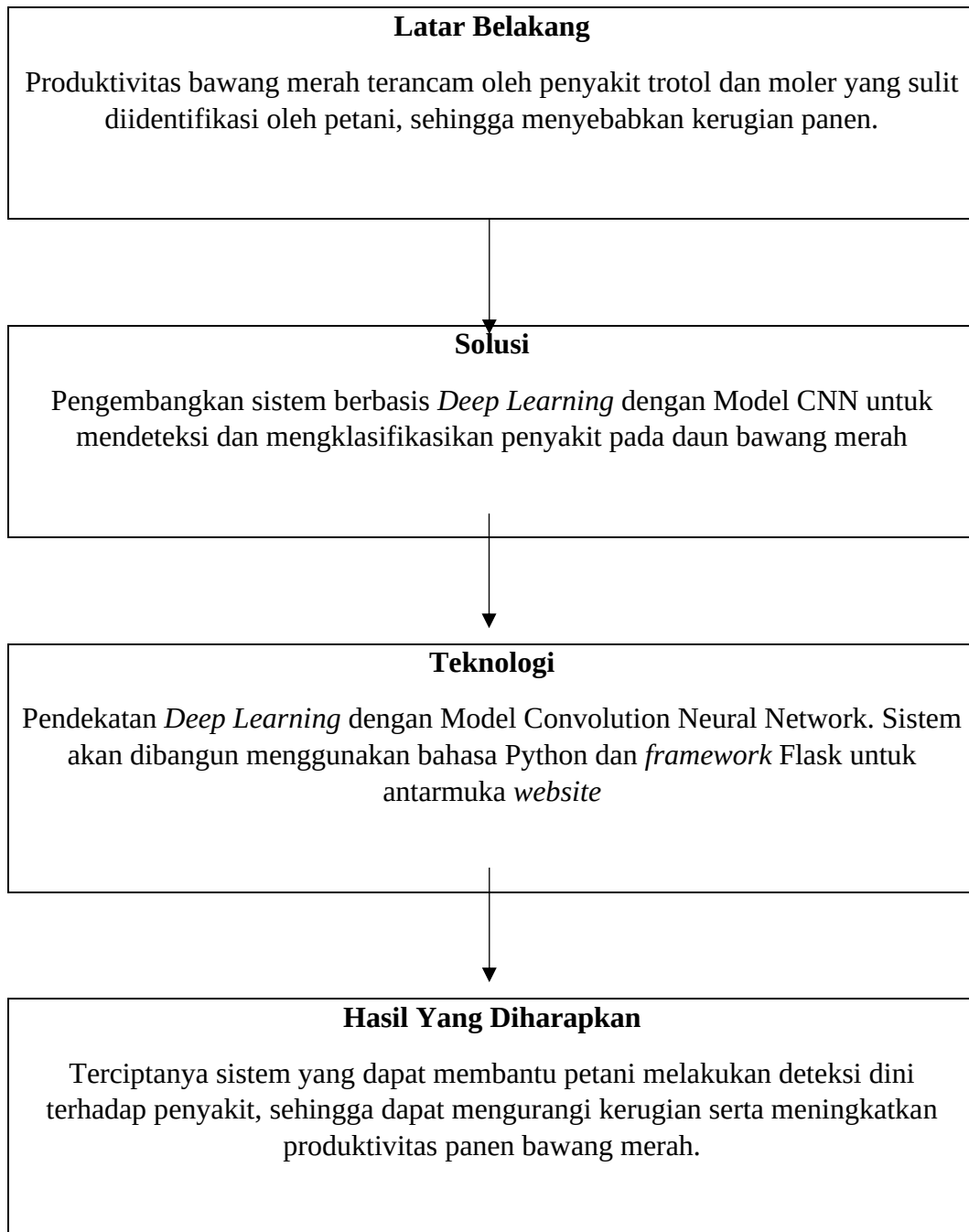
Penelitian sebelumnya sangat penting untuk digunakan sebagai acuan, pertimbangan, dan data pendukung dalam melaksanakan penelitian. Berikut ini adalah beberapa studi terdahulu yang telah dilakukan:

1. Azurah dkk (2023), dalam penelitiannya mengklasifikasikan penyakit daun bawang merah khususnya jenis bercak ungu dan moler pada 320 sampel citra menggunakan metode *Four Descriptor* (FD) yang dikombinasikan dengan Naïve Bayes dan *Covolutional Neural Network* (CNN). Hasil pengujian menunjukkan bahwa kombinasi FD-CNN mencapai akurasi tertinggi sebesar 98%, disusul oleh model CNN tanpa ekstraksi fitur sebesar 97%, serta FD-Gaussian Naïve Bayes sebesar 95%.
2. Pamungkas & Amrulloh (2025), menganalisis hasil klasifikasi penyakit daun bawang merah menggunakan Convolutional Neural Network (CNN) arsitektur Xception pada 240 sampel citra. Penelitian ini menerapkan preprocessing berupa cropping, resizing menjadi 224 x 224 piksel, serta augmentasi data. Hasil penelitian menunjukkan bahwa konfigurasi model terbaik menggunakan *batch size* 16 dan 100 *epoch* berhasil mencapai akurasi pelatihan sebesar 99,71% dan akurasi validasi sebesar 97,37%. Evaluasi lebih lanjut menggunakan *confusion matrix* pada 96 data uji menghasilkan tingkat akurasi sebesar 97% dengan 94 sampel terklasifikasi dengan benar.
3. Riqza Ardiansyah & Putra Pamungkas (2024), melakukan peneliti untuk mendeteksi penyakit tanaman bawang merah menggunakan algoritma *Support Vector Machine* (SVM) berbasis fungsi linear pada 250 sampel citra digital.

Penelitian ini menguji variasi resolusi citra serta berbagai rasio pembagian data *training* dan *testing*. Hasil eksperimen menunjukkan bahwa perubahan ukuran gambar tidak mempengaruhi signifikan terhadap performa model, dengan hasil kinerja tertinggi di capai pada rasio test size 27% dan 30%, yaitu menghasilkan akurasi sebesar 79%, precision 79%, dan F1-score 79%.

Berdasarkan beberapa penelitian terdahulu, dapat disimpulkan bahwa penerapan algoritma berbasis deep learning, khususnya Convolutional neural network (CNN) baik dengan kombinasi fourier descriptor maupun arsitektur Xception, terbukti menghasilkan performa dan akurasi klasifikasi yang jauh lebih tinggi di bandingkan algortima machine learning konvensional seperti support vector machine yang hanya mencapai akurasi 79%.

## B. Kerangka Berpikir



**Gambar 2. 10** Kerangka pikir

## BAB III

### METODE PENELITIAN

#### A. Jenis Penelitian

Jenis Penelitian ini menggunakan metode penelitian kuantitatif. Penelitian kuantitatif bertujuan merancang, melatih, serta mengevaluasi kinerja model *Convolution Neural Network* (CNN) dalam mendeteksi dan mengklasifikasikan penyakit trotol dan penyakit moler pada citra daun bawang merah. Pengumpulan dan pengolahan data dilakukan secara terstruktur melalui berbagai tahapan ilmiah, seperti studi literatur, akusisi citra daun bawang merah, kurasi dan pelabelan data secara manual bersama ahli, hingga pembagian dataset untuk proses pelatihan (*training*), validasi, dan pengujian model.

#### B. Lokasi dan Waktu

Penelitian ini akan dilaksanakan di Balai Penyuluhan Pertanian (BPP) Kecamatan Anggeraja Kabupaten Enrekang Sulawesi Selatan. Diperkirakan, penelitian ini akan berlangsung selama tiga bulan, yang mencakup tahapan studi literatur dan pengumpulan data, analisis kebutuhan, perancangan, pengembangan, serta evaluasi dan pengujian sistem.

**Tabel 3. 1** Lokasi dan Waktu

| No | Uraian Kegiatan                     | Bulan 2026 |      |      |
|----|-------------------------------------|------------|------|------|
|    |                                     | Mei        | Juni | Juli |
| 1  | Studi literatur & pengumpulan data  |            |      |      |
| 2  | Pra-pemrosesan & Analisis kebutuhan |            |      |      |

**Lanjutan Tabel 3.1**

| No | Uraian Kegiatan                  | Bulan 2026 |      |      |
|----|----------------------------------|------------|------|------|
|    |                                  | Mei        | Juni | Juli |
| 3  | Perancangan Sistem               |            |      |      |
| 4  | Implementasi dan Pelatihan model |            |      |      |
| 5  | Evaluasi dan Pengujian model     |            |      |      |
| 6  | Hasil                            |            |      |      |

### **C. Alat dan Bahan**

Untuk mendukung perancangan penerapan Klasifikasi Penyakit Bawang Merah Dengan Pendekatan Deep Learning, berikut adalah spesifikasi alat dan bahan yang digunakan dalam penelitian ini

#### **1. Alat**

##### **a. Perangkat Keras (*Hardware*)**

- 1) *Personal Computer* (PC) atau Laptop AsusVivobook
  - a) 64 bit AMD Ryzen 3 3200U with Radeon Vega Mobile Gfx 2.60 Ghz
  - b) Kapasitas RAM 8 GB
  - c) Perangkat *mouse* dan *keyboard* standar
- 2) Printer Dokumen untuk mencetak laporan

##### **b. Perangkat Lunak (*Software*)**

- 1) Sistem Operasi Windows 11
- 2) Bahasa Pemrograman Python
- 3) Teks Editor *Visual studio Code*

## 2. Bahan

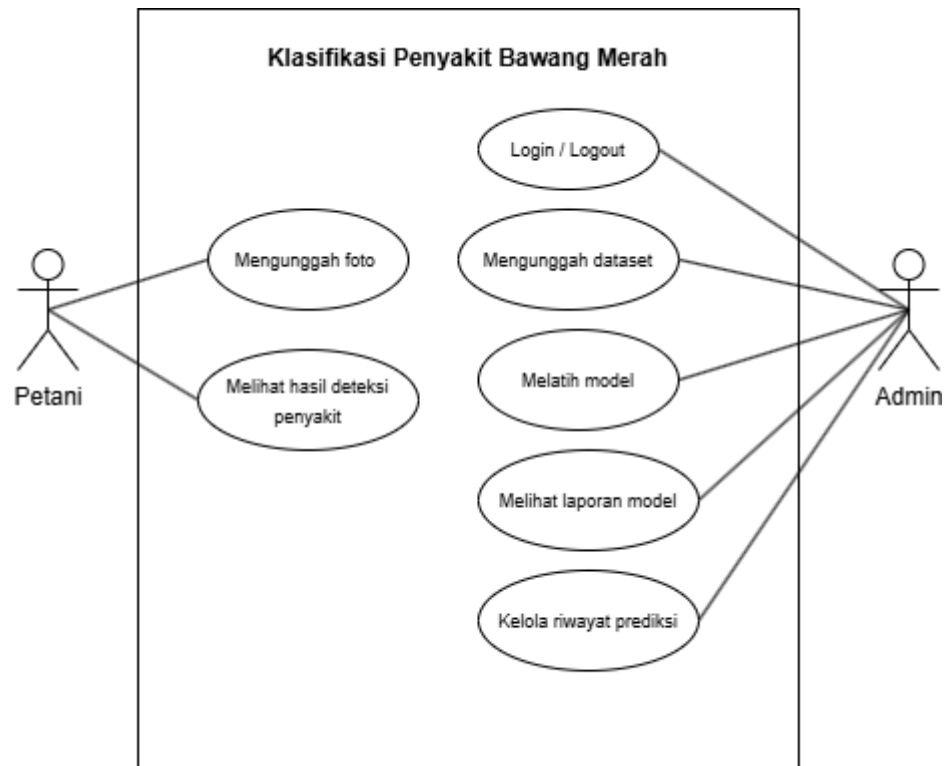
Bahan yang digunakan dalam penelitian ini berupa dataset citra daun dan umbi bawang merah, dengan jumlah total 2000 citra. Dataset dikelompokkan ke dalam empat kelas, yaitu Sehat, Trotol, Moler, dan Bukan Bawang, dengan jumlah citra pada tiap kelas diusahakan proporsional agar tidak terjadi ketidakseimbangan kelas (*class imbalance*) yang signifikan.

Citra diperoleh melalui tiga jalur akuisisi, yaitu dokumentasi lapangan berupa pengambilan foto langsung pada tanaman bawang merah di lahan pertanian wilayah Balai Penyuluhan Pertanian (BPP) Kecamatan Anggeraja, Kabupaten Enrekang; sumber terbuka (*open-source*) berupa dataset publik dari repositori atau publikasi ilmiah.

Setiap citra disimpan dalam format digital umum (JPEG/PNG) dan diberi label sesuai kategori penyakit melalui proses kurasi dan pelabelan manual dengan bantuan ahli atau referensi terpercaya. Dataset yang telah bersih dan berlabel selanjutnya dibagi menjadi data pelatihan, validasi, dan pengujian dengan proporsi masing-masing sekitar 70%, 15%, dan 15%, sebagaimana dijelaskan pada sub-bab Teknik Pengumpulan Data.

## D. Rancangan Sistem

### 1. Use Case Diagram



**Gambar 3. 1** Use Case Diagram

Pada tahap perancangan sistem, dilakukan pemodelan alur kerja yang akan dibangun. Sistem yang diusulkan direpresentasikan melalui *Use Case Diagram*.

**Tabel 3. 2** Deskripsi Use Case Diagram

| No | Nama Use Case                  | Deskripsi                                                                                         |
|----|--------------------------------|---------------------------------------------------------------------------------------------------|
| 1  | Mengunggah foto                | Petani dapat mengunggah foto daun bawang merah yang ingin diperiksa.                              |
| 2  | Melihat hasil deteksi penyakit | Petani dapat melihat hasil identifikasi jenis penyakit dari foto yang telah diproses oleh sistem. |

Lanjutan Tabel 3. 2

| No | Nama Use Case         | Deskripsi                                                                                                                                                                            |
|----|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3  | Mengunggah dataset    | Admin dapat mengunggah dan mengelolah dataset citra daun bawang merah ke dalam sistem, yang nantinya akan dipelajari oleh model dalam proses pelatihan.                              |
| 4  | Melatih model         | Sistem menampilkan visualisasi <i>feature maps</i> hasil operasi konvolusi guna menunjukkan proses ekstraksi fitur visual yang dilakukan oleh model saat melakukan deteksi penyakit. |
| 5  | Melihat laporan model | Admin melakukan latihan pada model Convolutional Neural Network (CNN) menggunakan dataset citra yang telah tersimpan di dalam sistem                                                 |

## 2. Desain UI yang diusulkan

Klasifikasi Penyakit Bawang Merah

dengan pendekatan deep learning — mode detail (wireframe)

1

Input Gambar

2

Konvolusi

3

ReLU

4

Pooling

5

Global Average Pooling

6

Fully Connected

7

Hasil Deteksi

Unggah Gambar Daun

Unggah foto daun bawang merah untuk dianalisis menggunakan model CNN. Pastikan gambar cukup terang dan fokus pada daun.

Pilih Gambar

Seret & Lepas foto di sini

Ambil Foto Langsung

Pratinjau

Nama file

Ukuran

Resolusi

PRA-PROSES OTOMATIS

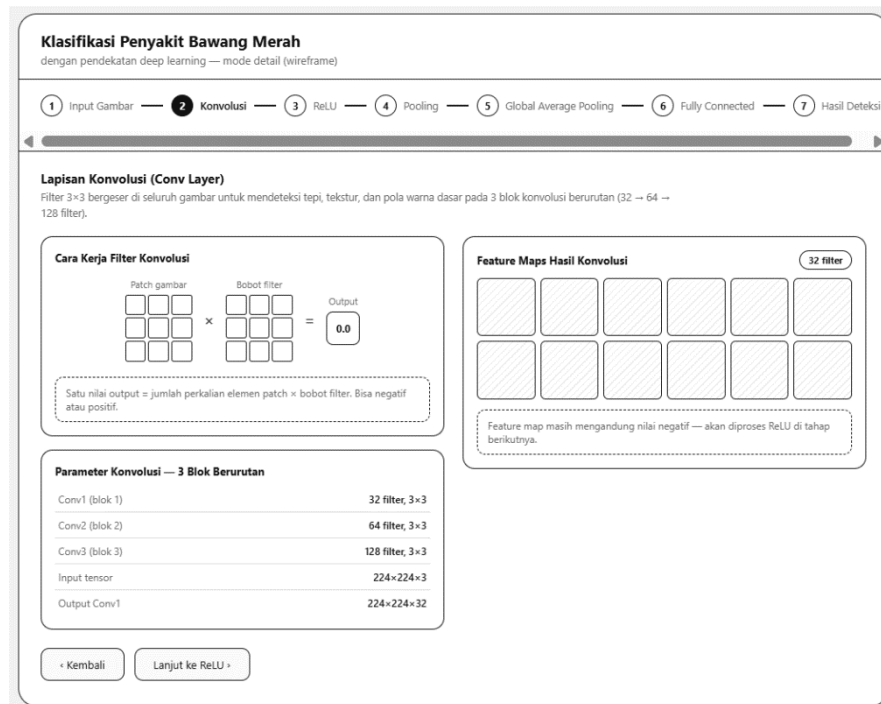
ubah ukuran ke 224 x 224

Normalisasi ukuran pixel [0,1]

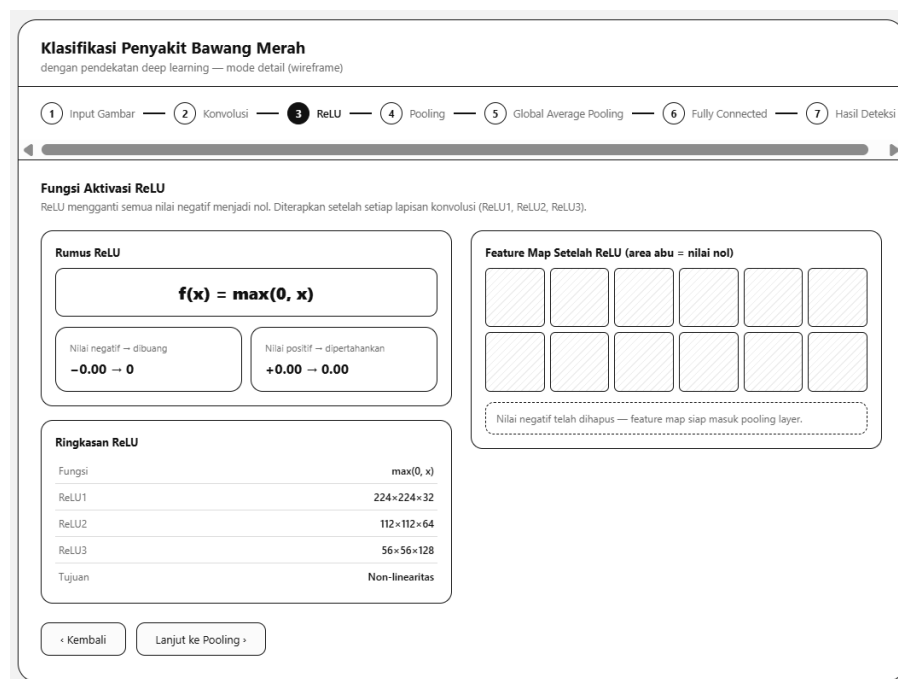
Proses Gambar

Gambar 3. 2 Tampilan Input Gambar

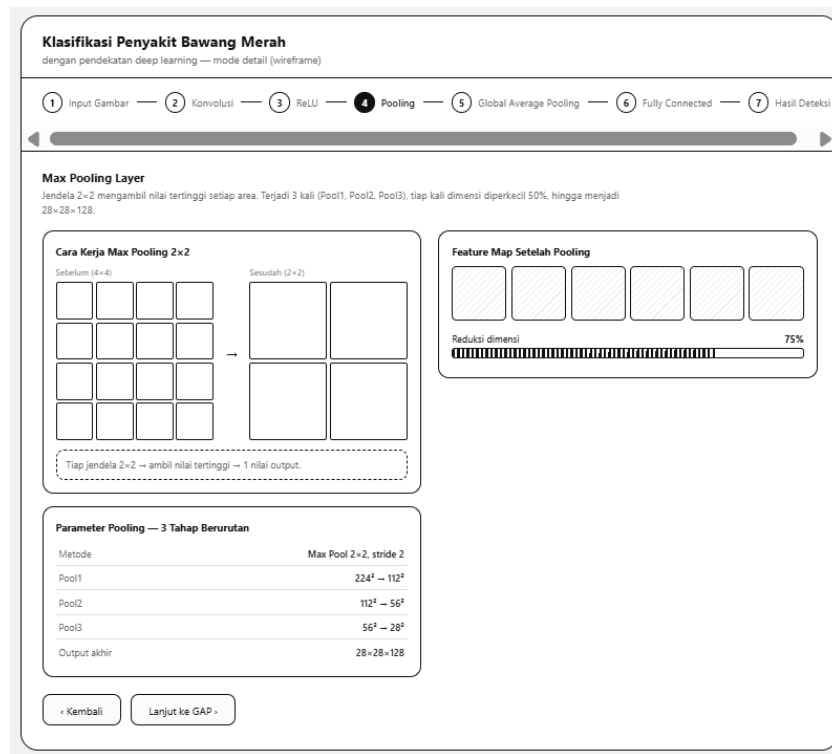
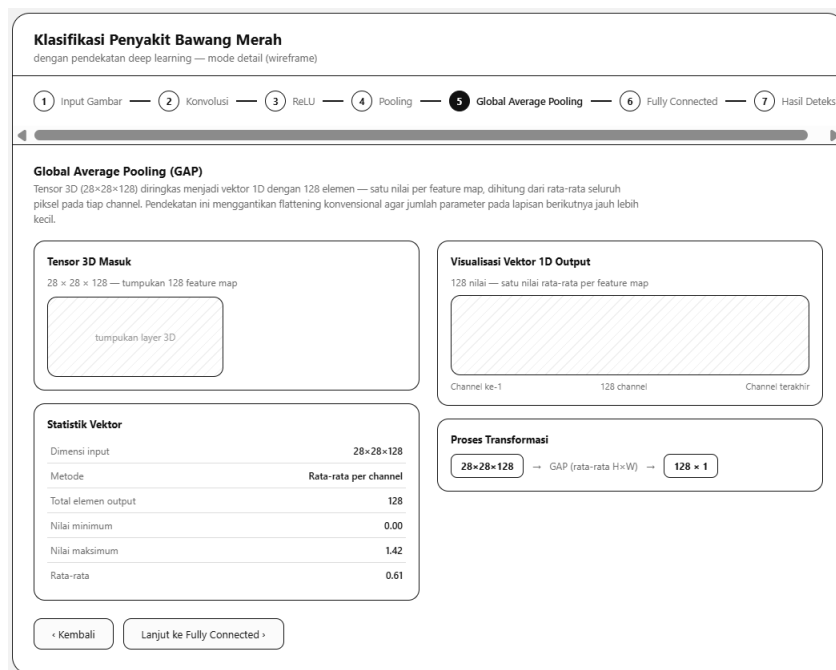


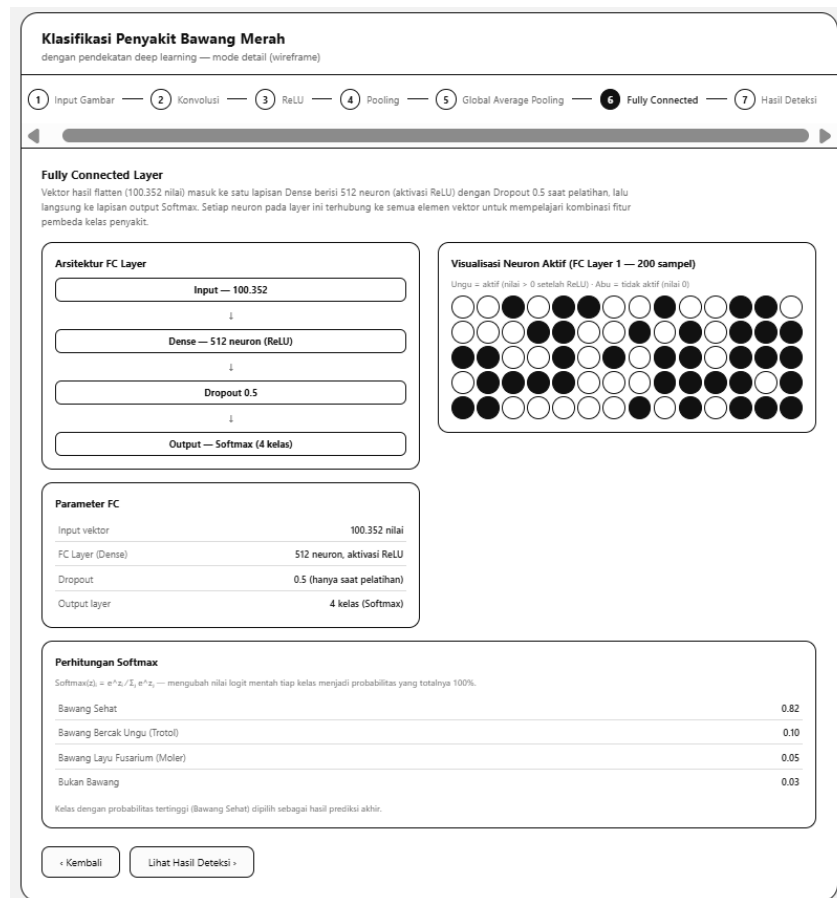


Gambar 3. 3 Tampilan Lapisan Konvolusi



Gambar 3. 4 Tampilan Fungsi Aktivasi ReLU

Gambar 3. 5 Tampilan *Pooling Layer*Gambar 3. 6 Tampilan *Global Average Pooling*



**Gambar 3. 7** Tampilan Fully Connected Layer



**Gambar 3. 8** Tampilan hasil deteksi

### E. Teknik Pengumpulan Data

Penelitian ini mengandalkan teknik pengumpulan data yang berfokus pada pembentukan dataset citra yang terstruktur serta teranotasi dengan baik. Tahap ini merupakan fondasi utama, mengingat kualitas dan kuantitas data akan menentukan tingkat akurasi serta kemampuan generalisasi model Convolutional Neural Network (CNN) yang dikembangkan. Untuk mencapai hal tersebut, berikut adalah tahapan pengumpulan data yang dilakukan:

1. Studi literatur: Tahapan ini dilakukan untuk mengkaji referensi ilmiah, teori pendukung, serta penelitian terdahulu yang relevan terkait pengembangan model

*machine learning* menggunakan arsitektur *Convolutional Neural Network* (CNN).

2. Observasi: Observasi dilakukan dengan mengamati dan mengumpulkan data secara langsung di lapangan, baik pada tanaman bawang merah yang sehat maupun yang terinfeksi penyakit.
3. Dataset Publik: Pengambilan dataset sekunder dilakukan melalui *platform* repositori publik seperti kaggle guna menambah jumlah variasi *dataset*.
4. Strukturisasi dan Pengujian Dataset

Dataset yang telah bersih dan berlabel selanjutnya diorganisir serta dikelompokkan ke dalam dua tahapan pengujian, yaitu:

a. Dataset Utama (Pengujian Internal Model)

Dataset utama dibagi secara proporsional menjadi 3 sub-set untuk keperluan pelatihan dan evaluasi kinerja internal model CNN:

- 1) Data Data Pelatihan (*Training Data*): Merupakan bagian terbesar dari dataset 80% yang digunakan untuk melatih model CNN agar mampu mengenali pola visual dari setiap kelas penyakit.
- 2) Data Validasi (*Validation Data*): Sebagian kecil data 10% yang digunakan untuk memantau dan mengevaluasi performa model secara berkala selama pelatihan. Ini membantu dalam penyesuaian hyperparameter dan mencegah overfitting.
- 3) Data Pengujian Internal (*Testing Data*): Porsi data sebesar 10% dari total dataset utama yang digunakan untuk mengukur kinerja empiris awal

model (*Akurasi, Precision, Recall, F1-Score, dan Confusion Matrix*) setelah pelatihan selesai.

b. Data Uji Mandiri

Selain dataset utama, penelitian ini juga menyiapkan data uji baru yang belum pernah digunakan sebelumnya. Data ini dipakai untuk menguji kemampuan sistem web dalam mendeteksi foto daun bawang merah secara langsung di lapangan.

## **F. Teknik Analisis Data**

### **1. Pelatihan dan Validasi Model**

Model CNN dilatih menggunakan dataset citra daun bawang merah yang telah dibagi ke dalam subset data pelatihan dan data validasi dan data test dengan proporsi masing-masing 80% dan 10%. Selama proses pelatihan, jaringan mengekstraksi fitur spasial visual secara bertahap. Data validasi dievaluasi pada setiap *epoch* untuk memantau nilai *loss* dan akurasi guna mencegah *overfitting* serta memicu mekanisme *early stopping* dan penyesuaian *learning rate*.

### **2. Pengujian Kinerja Model**

Pengujian model dilakukan dalam dua skenario:

- a. Data Uji Internal: Menggunakan 15% sisa dataset (200 citra) yang tidak pernah dilibatkan dalam proses pelatihan untuk mengukur kemampuan generalisasi model secara objektif.
- b. Data Pengujian Mandiri: Menggunakan 40 citra baru yang disiapkan terpisah dari dataset utama untuk menguji ketahanan model pada variasi citra dunia nyata.

### 3. Evaluasi dengan Metrik Kuantitatif

Hasil pengujian dianalisis menggunakan metrik kuantitatif yang meliputi:

- a. Confusion Matrix: Untuk memvisualisasikan persebaran prediksi benar (*True Positive*) dan kesalahan klasifikasi (*False Positive/False Negative*) antar-kelas.
- b. Akurasi (*Accuracy*): Persentase total prediksi benar dibanding keseluruhan sampel data uji.
- c. *Precision*, *Recall*, dan *F1-Score*: Digunakan untuk menilai ketepatan prediksi dan kemampuan model dalam mengenali sampel positif pada tiap kelas (*Trotol*, *Moler*, *Sehat*, dan *Bukan Bawang*) secara seimbang.

### 4. Black-Box Testing

Selain evaluasi model kecerdasan buatan, penelitian ini menerapkan pengujian antarmuka sistem web menggunakan metode *Black-Box Testing*. Pengujian ini berfokus pada validasi fungsi antarmuka tanpa melihat struktur kode internal, meliputi pengujian alur unggah citra, pengujian fungsi tombol navigasi tahapan pipeline CNN (Konvolusi, ReLU, Pooling, Flatten, Fully Connected), pengujian penolakan berkas non-gambar, serta verifikasi kelancaran tampilan luaran sistem.

## BAB IV

### HASIL DAN PEMBAHASAN

#### A. Analisis Kebutuhan dan Rancangan

##### 1. Flowchart Sistem

Flowchart sistem dirancang untuk menggambarkan bagaimana pengguna, khususnya petani atau penyuluh pertanian, berinteraksi dengan aplikasi web sejak citra daun bawang merah dimasukkan hingga sistem mengeluarkan diagnosis beserta rekomendasi penanganan. Rancangan alur ini disusun agar proses tetap sederhana dari sisi pengguna, meskipun di baliknya terjadi beberapa tahap pemrosesan citra yang cukup kompleks.

gambar

Alur dimulai ketika pengguna membuka aplikasi melalui peramban (*browser*) dan memasukkan gambar daun bawang merah, baik melalui fitur unggah berkas (*drag and drop*) maupun tangkapan kamera secara *real-time*. Pada tahap ini sistem terlebih dahulu memeriksa format berkas yang diunggah hanya ekstensi *.jpg*, *.jpeg*, *.png*, *.bmp*, dan *.webp* yang diterima. Jika format tidak sesuai, misalnya pengguna secara tidak sengaja mengunggah berkas dokumen atau video, sistem akan menampilkan pesan peringatan dan menghentikan proses sebelum gambar sempat masuk ke tahap inferensi.



Perlu dibedakan antara validasi format ini dengan mekanisme penanganan gambar yang valid secara format namun bukan citra bawang merah (misalnya pengguna mengunggah foto tanah, gulma, atau tanaman lain). Untuk kasus ini, sistem tidak melakukan penolakan di tahap awal, melainkan tetap memproses gambar tersebut hingga ke tahap inferensi CNN. Kelas `non_bawang` sengaja disertakan sebagai salah satu dari empat target klasifikasi, sehingga ketika model mendeteksi bahwa objek pada citra tidak memiliki karakteristik visual bawang merah, keluaran sistem berupa label prediksi Non Bawang bukan berupa pesan kegagalan sistem. Pendekatan ini dipilih agar sistem tetap memberikan respons yang informatif kepada pengguna, alih-alih menolak begitu saja input yang secara teknis valid namun di luar cakupan objek yang dituju.

Setelah gambar dinyatakan lolos validasi format, sistem melakukan pra-pemrosesan otomatis berupa perubahan ukuran (*resizing*) citra menjadi  $224 \times 224$  piksel dan normalisasi nilai piksel ke rentang  $[0, 1]$  ( $\text{rescale} = 1./255$ ), menyesuaikan dengan spesifikasi input yang dibutuhkan arsitektur MobileNetV2. Citra yang telah dinormalisasi kemudian diteruskan ke model *Deep Learning* untuk melalui tahap ekstraksi fitur dan klasifikasi, meliputi lapisan *Global Average Pooling* (GAP), *Batch Normalization*, *Dropout*, *Dense Layer* 128 neuron, hingga *Output Layer* berfungsi aktivasi *Softmax* yang menghasilkan probabilitas untuk masing-masing dari empat kelas target.

Pada tahap akhir, sistem menampilkan label hasil prediksi Moler, Non Bawang, Sehat, atau Trotol beserta nilai probabilitas kepercayaan (*confidence score*) kepada pengguna. Bagi citra yang terdeteksi mengalami gejala penyakit, sistem turut

menampilkan rekomendasi penanganan sekaligus menyediakan opsi unduh laporan hasil deteksi dalam format PDF yang dapat langsung dimanfaatkan oleh petani di lapangan, tanpa menyimpan riwayat prediksi ke basis data.

## 2. Alur Kerja CNN

Arsitektur model yang digunakan dalam penelitian ini, diberi nama *BawangMerah\_MobileNetV2*, dibangun menggunakan pendekatan *Transfer Learning* yaitu memanfaatkan model yang telah dilatih sebelumnya pada dataset besar (ImageNet) sebagai dasar ekstraksi fitur, kemudian dikombinasikan dengan lapisan klasifikasi baru yang disesuaikan untuk kebutuhan spesifik penelitian, yaitu mengenali empat kondisi bawang merah. Pendekatan ini dipilih karena jumlah data yang tersedia dalam penelitian ini (2.000 citra) relatif terbatas untuk melatih CNN dari awal (*from scratch*), sehingga memanfaatkan bobot yang sudah terlatih pada jutaan gambar dapat mempercepat konvergensi sekaligus meningkatkan akurasi model.

Citra masukan yang telah melalui tahap pra-pemrosesan diterima dalam bentuk matriks berdimensi  $224 \times 224 \times 3$ , sesuai dengan format input standar MobileNetV2. Pada tahap *Base Model*, seluruh bobot MobileNetV2 hasil pelatihan pada ImageNet **dibekukan** (*trainable = False*), artinya lapisan-lapisan ini tidak diperbarui selama proses pelatihan berlangsung. Keputusan ini diambil agar kemampuan model dalam mengenali fitur-fitur dasar seperti tepi, garis, dan tekstur warna yang telah terbentuk dari pelatihan pada jutaan

gambar sebelumnya tetap stabil dan tidak rusak (*catastrophic forgetting*) akibat proses pelatihan ulang pada dataset yang jauh lebih kecil.

Keluaran dari *base model* berupa *feature map* tiga dimensi kemudian diringkas melalui lapisan *Global Average Pooling* (GAP), yang mengubah setiap kanal fitur menjadi satu nilai rata-rata sehingga dihasilkan vektor satu dimensi. Pendekatan GAP dipilih dibanding *Flatten* konvensional karena mampu mengurangi jumlah parameter secara signifikan, sehingga risiko *overfitting* dapat ditekan pertimbangan yang cukup relevan mengingat ukuran dataset penelitian ini tidak terlalu besar.

Vektor fitur hasil GAP selanjutnya dinormalisasi melalui lapisan *Batch Normalization* untuk menstabilkan distribusi aktivasi dan mempercepat konvergensi pelatihan, sebelum diteruskan ke *Dropout Layer* pertama dengan tingkat *drop rate* 0,3 (30%) yang secara acak menonaktifkan sebagian neuron selama pelatihan sebagai bentuk regularisasi terhadap *overfitting*. Fitur yang telah diregularisasi kemudian diproses oleh *Dense Layer* berisi 128 neuron dengan fungsi aktivasi ReLU untuk mempelajari kombinasi fitur yang lebih kompleks dan spesifik terhadap karakteristik penyakit bawang merah, sebelum melalui *Dropout Layer* kedua (0,2) sebagai lapisan regularisasi tambahan.

Sebagai lapisan penutup, *Dense Output Layer* terdiri atas 4 neuron dengan fungsi aktivasi *Softmax*, yang mengonversi nilai keluaran mentah (*logits*) menjadi distribusi probabilitas untuk keempat kelas target *moler*, *non\_bawang*, *sehat*, dan *trotol* di mana kelas dengan nilai probabilitas tertinggi ditetapkan sebagai hasil prediksi akhir sistem.

## B. Implementasi Tools Pengembangan

Pengembangan sistem klasifikasi penyakit bawang merah ini sepenuhnya memanfaatkan ekosistem perangkat lunak berbasis Python, mengingat bahasa ini memiliki dukungan pustaka *machine learning* yang matang serta kemudahan integrasi antara model *Deep Learning* dengan aplikasi web. Berikut dipaparkan peran masing-masing *tools* dalam keseluruhan sistem, mulai dari *backend* API, model *Deep Learning*, hingga pemrosesan citra.

### 1. Python & FastApi

Sisi *backend* aplikasi dibangun menggunakan **FastAPI** (`src/main_api.py`), sebuah *framework* web Python modern berbasis standar ASGI (*Asynchronous Server Gateway Interface*). Pemilihan FastAPI didasari kebutuhan untuk menangani proses inferensi model yang relatif memakan waktu tanpa memblokir permintaan lain yang masuk secara bersamaan, sehingga dukungan native terhadap operasi asinkronus (`async/await`) menjadi pertimbangan utama dibanding *framework* sinkron konvensional seperti Flask.

FastAPI dimanfaatkan untuk membangun dua *endpoint* utama. *Endpoint* `/predict` menerima unggahan citra tanaman dari pengguna dan mengembalikan hasil klasifikasi dalam format JSON, sementara *endpoint* `/extract-layers` yang terhubung dengan modul `src/layer_extraction.py` bertugas mengekstraksi dan menyajikan representasi internal jaringan CNN secara *real-time* untuk setiap citra yang diproses. Modul ini bekerja dengan membangun *multi-output model* dari arsitektur MobileNetV2, yaitu model Keras yang sama namun dikonfigurasi untuk

mengeluarkan nilai aktivasi dari beberapa lapisan sekaligus dalam satu kali proses `predict()` meliputi lapisan konvolusi pertama, lapisan ReLU pertama, lapisan *pooling* awal (`block_1_project_bn`), hingga lapisan *Global Average Pooling* dan *Dense* 128 neuron pada bagian *classification head*.

Setiap *feature map* yang diekstraksi kemudian dikonversi menjadi citra visual melalui proses normalisasi nilai piksel dan pewarnaan (*tinting*) yang berbeda untuk tiap tahap nuansa ungu-kebiruan untuk lapisan konvolusi, hijau *mint* untuk lapisan aktivasi ReLU, dan kuning keemasan untuk lapisan *pooling* sehingga pengguna dapat membedakan tiap tahap pemrosesan secara visual. Selain representasi gambar, modul ini juga menghitung statistik kuantitatif di setiap tahap, seperti nilai minimum, maksimum, rata-rata, standar deviasi, serta persentase neuron aktif pasca-ReLU, yang seluruhnya ditampilkan pada antarmuka web sebagai bagian dari visualisasi *pipeline* CNN (lihat Bagian E).

## 2. Tensorflow & Keras

**TensorFlow** (versi 2.x) beserta pustaka Keras digunakan sebagai fondasi utama dalam perancangan, pelatihan, dan pengujian model CNN (`src/train_model.py`). Proses pelatihan dipandu oleh *optimizer* Adam dengan *learning rate*  $10^{-4}$  ( $1e-4$ ) dan fungsi kerugian *Categorical Crossentropy*, yang sesuai untuk permasalahan klasifikasi multi-kelas dengan label berbentuk *one-hot encoding*.

Untuk menjaga proses pelatihan tetap efisien dan menghindari *overfitting*, pelatihan dilengkapi tiga mekanisme *callback*. *EarlyStopping* menghentikan pelatihan

apabila nilai `val_loss` tidak menunjukkan perbaikan selama 5 epoch berturut-turut (`patience=5`), sehingga model tidak terus dilatih ketika sudah mengalami stagnasi atau justru mulai *overfitting* terhadap data latih. `ModelCheckpoint` secara otomatis menyimpan bobot model dengan `val_accuracy` tertinggi ke berkas `models/best_bawang_model.h5`, memastikan model akhir yang digunakan bukan model dari epoch terakhir, melainkan model dengan performa validasi terbaik sepanjang proses pelatihan. Sementara itu, `ReduceLROnPlateau` menurunkan *learning rate* sebesar faktor 0,2 apabila `val_loss` mengalami stagnasi selama 3 epoch, membantu model keluar dari kondisi *plateau* dan mencapai konvergensi yang lebih halus.

### 3. NumPy & Pillow

Pemrosesan citra pada sisi *backend* memanfaatkan dua pustaka pendukung. Pillow (PIL) digunakan untuk membuka berkas citra yang diunggah pengguna, mengubah ukurannya menjadi  $224 \times 224$  piksel sesuai kebutuhan input model, serta melakukan konversi mode warna ke RGB langkah yang diperlukan karena tidak semua gambar unggahan pengguna berada dalam format warna yang seragam. Hasil citra yang telah diproses Pillow kemudian dikonversi menjadi array numerik menggunakan NumPy, termasuk konversi tipe data ke float32 dan normalisasi nilai piksel melalui pembagian dengan 255,0, agar rentang nilai yang diterima model konsisten dengan data yang digunakan saat pelatihan.

## C. Explore Dataset

## 1. Deskripsi Dataset

Dataset penelitian ini disusun dari citra daun dan umbi tanaman bawang merah (*Allium ascalonicum* L.) yang dikumpulkan untuk merepresentasikan kondisi-kondisi paling umum ditemui di lapangan, ditambah satu kategori objek kontrol untuk menguji kemampuan sistem menolak masukan di luar cakupan bawang merah. Secara keseluruhan, dataset terbagi ke dalam empat kelas kategori sebagaimana didefinisikan pada berkas `models/class_indices.json`:

1. Moler (moler): Citra tanaman bawang merah yang terserang penyakit layu fusarium (*Fusarium oxysporum*), ditandai dengan daun melengkung atau terpilin, menguning, serta terjadinya pembusukan pada akar maupun umbi. Penyakit ini dipilih sebagai salah satu kelas target karena tergolong penyakit yang paling merugikan secara ekonomi bagi petani bawang merah di Indonesia.
2. Bukan Bawang (non\_bawang): Citra objek latar belakang, tanah, gulma, atau tanaman lain yang berfungsi sebagai kontrol negatif, agar sistem mampu membedakan antara objek bawang merah dan objek di luar cakupan penelitian, bukan sekadar memaksakan klasifikasi ke salah satu dari tiga kelas penyakit/sehat.
3. Sehat (sehat): Citra daun bawang merah dalam kondisi sehat, tanpa menunjukkan gejala infeksi patogen apa pun, digunakan sebagai baseline pembandingan terhadap dua kelas penyakit.

4. Trotol (trotol): Citra daun bawang merah yang terserang penyakit bercak ungu (*Alternaria porri*), ditandai dengan munculnya bercak berbentuk oval berwarna keabu-abuan hingga keunguan pada permukaan daun.

## 2. Deskripsi Data Penelitian

Penelitian ini menggunakan total 2.000 sampel citra digital yang kemudian dibagi secara terstruktur menggunakan modul `src/split_dataset.py` ke dalam tiga subset: 70% data latih (*training*), 15% data validasi (*validation*), dan 15% data uji (*testing*). Pembagian dengan rasio ini dipilih untuk memastikan model memperoleh data latih yang cukup banyak untuk mempelajari pola visual tiap kelas, sekaligus tetap menyisakan porsi data yang memadai untuk memantau performa selama pelatihan (*validasi*) dan mengukur generalisasi model terhadap data yang benar-benar belum pernah dilihat sebelumnya (*pengujian*). Rincian distribusi jumlah sampel per kelas disajikan pada Tabel 4.1.

| Kelas Kategori | Data Training (70%) | Data Validasi (15%) | Data Testing (15%) | Total Sampel |
|----------------|---------------------|---------------------|--------------------|--------------|
| Moler          | 280                 | 60                  | 60                 | 400          |
| Bukan bawang   | 280                 | 60                  | 60                 | 400          |
| Sehat          | 420                 | 90                  | 90                 | 600          |
| Trotol         | 420                 | 90                  | 90                 | 600          |
| Total Overall  | 1400                | 300                 | 300                | 2000         |

Perlu dicatat bahwa jumlah sampel antar kelas tidak sepenuhnya seimbang kelas Sehat dan Trotol masing-masing memiliki 600 sampel, sementara kelas Moler dan Bukan Bawang masing-masing 400 sampel. Ketidakseimbangan ini mengikuti proporsi ketersediaan data di lapangan, di mana kondisi daun sehat dan gejala



bercak ungu (trotol) relatif lebih mudah ditemukan dan didokumentasikan dibanding kasus layu fusarium yang lebih jarang teramati.

## **D. Evaluasi Performa Model**

Evaluasi performa model bertujuan untuk mengukur secara kuantitatif tingkat keandalan arsitektur Convolutional Neural Network (CNN) berbasis Transfer Learning MobileNetV2 dalam mengklasifikasikan citra penyakit bawang merah (*Allium ascalonicum* L.). Tahapan evaluasi meliputi tiga bagian utama, yaitu (1) analisis grafik akurasi dan loss selama proses pelatihan, (2) perhitungan Confusion Matrix beserta metrik kuantitatif Precision, Recall, dan F1-Score pada data uji, serta (3) simulasi perhitungan matematis manual terhadap operasi-operasi internal jaringan saraf konvolusi.

### **1. Plotting Akurasi dan Loss**

Proses pelatihan model dijalankan dengan memantau perkembangan nilai accuracy dan loss pada subset data training dan validation di setiap epoch, guna mengetahui tingkat konvergensi serta mendeteksi kemungkinan terjadinya overfitting maupun underfitting selama pelatihan berlangsung.

Grafik pelatihan yang dihasilkan oleh modul `src/train_model.py` dan disimpan pada berkas `reports/training_history.png` menunjukkan bahwa nilai loss mengalami penurunan secara konsisten seiring bertambahnya jumlah epoch, baik pada data training maupun data validation. Sejalan dengan itu, nilai accuracy menunjukkan tren peningkatan dan mencapai titik konvergensi pada tingkat yang tinggi tanpa terjadi kesenjangan (gap) yang signifikan antara kurva training dan validation. Kondisi ini mengindikasikan bahwa model mampu mempelajari pola fitur citra

dengan baik tanpa mengalami overfitting yang berarti, yang didukung pula oleh penerapan mekanisme regularisasi berupa Dropout Layer dan callback EarlyStopping selama proses pelatihan.

## 2. Confusion Matrix Dan Metrik Kuantitatif

Evaluasi performa klasifikasi dilakukan secara empiris terhadap 300 data uji (test set) yang sama sekali tidak dilibatkan selama proses pelatihan model, sebagaimana diimplementasikan pada modul `src/evaluate_model.py`. Pengujian pada data yang belum pernah dilihat oleh model ini penting dilakukan untuk memperoleh gambaran objektif mengenai kemampuan generalisasi model terhadap data baru.

| Label Aktual \ Label Prediksi | Moler | Bukan Bawang | Sehat | Trotol | Total Aktual |
|-------------------------------|-------|--------------|-------|--------|--------------|
| Moler                         | 55    | 0            | 2     | 3      | 60           |
| Bukan Bawang                  | 0     | 60           | 0     | 0      | 60           |
| Sehat                         | 2     | 0            | 85    | 3      | 90           |
| Trotol                        | 1     | 0            | 6     | 83     | 90           |
| Total Prediksi                | 58    | 60           | 93    | 89     | 300          |

Berdasarkan Confusion Matrix pada Tabel 4.2, dapat dihitung nilai Precision, Recall, dan F1-Score untuk masing-masing kelas menggunakan Persamaan (1), (2), dan (3) berikut:

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1-Score = 2 \times (Precision \times Recall) / (Precision + Recall)$$

dengan TP (True Positive) adalah jumlah sampel yang diprediksi benar sesuai kelas aktualnya, FP (False Positive) adalah jumlah sampel kelas lain yang keliru

diprediksi sebagai kelas tersebut, dan FN (False Negative) adalah jumlah sampel kelas tersebut yang keliru diprediksi sebagai kelas lain. Hasil perhitungan metrik kuantitatif per kelas disajikan pada Tabel 4.3

| Kelas Kategori                      | Precision | Recall  | F1-Score | Jumlah Sampel (Support) |
|-------------------------------------|-----------|---------|----------|-------------------------|
| Moler                               | 94,83%    | 91,67%  | 93,22%   | 60                      |
| Bukan Bawang                        | 100,00%   | 100,00% | 100,00%  | 60                      |
| Sehat                               | 91,40%    | 94,44%  | 92,90%   | 90                      |
| Trotol                              | 93,26%    | 92,22%  | 92,74%   | 90                      |
| Rata-rata Makro (Macro Avg)         | 94,87%    | 94,58%  | 94,71%   | 300                     |
| Rata-rata Tertimbang (Weighted Avg) | 94,36%    | 94,33%  | 94,33%   | 300                     |

Berdasarkan hasil pengujian pada Tabel 4.2 dan Tabel 4.3, diperoleh beberapa temuan sebagai berikut:

- Akurasi Keseluruhan (Overall Accuracy) berhasil mengklasifikasikan 283 dari 300 data uji dengan benar, sehingga mencapai tingkat akurasi keseluruhan sebesar 94,33%. Nilai ini menunjukkan bahwa model memiliki kemampuan generalisasi yang tinggi terhadap data yang belum pernah dipelajari sebelumnya.
- Kelas Bukan Bawang mencapai nilai Precision, Recall, dan F1-Score sebesar 100,00%, yang berarti seluruh 60 citra kontrol non-bawang berhasil dikenali secara sempurna tanpa satu pun kesalahan klasifikasi. Hal ini menunjukkan bahwa model mampu membedakan secara tegas antara objek daun bawang merah dengan objek latar belakang, tanah, gulma, maupun tanaman lain.
- Kesalahan Klasifikasi (Misclassification) dari total 300 data uji, terdapat 17 citra yang mengalami kesalahan prediksi. Kesalahan paling banyak terjadi

antara kelas Sehat dan Trotol, yaitu 6 sampel Trotol yang terprediksi sebagai Sehat dan 3 sampel Sehat yang terprediksi sebagai Trotol. Kesalahan ini diduga dipengaruhi oleh kemiripan visual pada citra daun dengan gejala infeksi *Alternaria porri* tingkat awal, di mana bercak ungu pada daun belum tampak jelas sehingga menyerupai daun sehat.

- d. Rata-rata Makro dan Tertimbang dengan Nilai Macro Average F1-Score sebesar 94,71% dan Weighted Average F1-Score sebesar 94,33% menunjukkan konsistensi performa model pada seluruh kelas, baik yang memiliki jumlah sampel seimbang (Moler dan Bukan Bawang) maupun yang memiliki jumlah sampel lebih banyak (Sehat dan Trotol).

### 3. Perhitungan Manual CNN

Untuk memberikan pemahaman yang lebih mendalam mengenai proses komputasi internal jaringan saraf konvolusi, berikut disajikan simulasi perhitungan matematis secara manual pada tiap tahapan pemrosesan citra, mulai dari operasi konvolusi hingga fungsi aktivasi Softmax.

#### a. Operasi Konvolusi (Convolutional Layer)

Misalkan diambil satu matriks sub-citra masukan berukuran  $3 \times 3$  (I) dan satu kernel filter berukuran  $3 \times 3$  (K) dengan nilai bias  $b = 0,5$  sebagai berikut:

$$I = \begin{bmatrix} 120 & 150 & 180 \\ 100 & 130 & 160 \\ 90 & 110 & 140 \end{bmatrix} \quad K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Sebelum dikonvolusi, nilai piksel pada matriks I terlebih dahulu dinormalisasi dengan cara membagi setiap elemen dengan 255 (rescale = 1/255), sehingga diperoleh matriks ternormalisasi sebagai berikut:

$$I_{norm} = \begin{bmatrix} 0,4706 & 0,5882 & 0,7059 \\ 0,3922 & 0,5098 & 0,6275 \\ 0,3529 & 0,4314 & 0,5490 \end{bmatrix}$$

Operasi konvolusi dilakukan dengan mengalikan setiap elemen matriks  $I_{norm}$  dengan elemen kernel K pada posisi bersesuaian (element-wise multiplication), kemudian dijumlahkan seluruhnya dan ditambahkan dengan nilai bias b, sehingga menghasilkan satu nilai keluaran  $S(1,1)$ :

$$S(1,1) = (0,4706 \times 1) + (0,5882 \times 0) + (0,7059 \times -1) + (0,3922 \times 1) + (0,5098 \times 0) + (0,6275 \times -1) + (0,3529 \times 1) + (0,4314 \times 0) + (0,5490 \times -1) + 0,5$$

$$S(1,1) = (0,4706 - 0,7059) + (0,3922 - 0,6275) + (0,3529 - 0,5490) + 0,5$$

$$S(1,1) = -0,2353 - 0,2353 - 0,1961 + 0,5 = -0,1667$$

Nilai -0,1667 tersebut merupakan hasil konvolusi mentah (raw feature) yang selanjutnya akan diteruskan ke fungsi aktivasi ReLU.

#### b. Fungsi Aktivasi ReLU (Rectified Linear Unit)

Fungsi aktivasi ReLU digunakan untuk mengubah seluruh nilai keluaran yang bersifat negatif menjadi nol, sehingga hanya mempertahankan nilai aktivasi yang bersifat positif, sebagaimana dirumuskan pada Persamaan (4):

$$f(x) = \max(0, x)$$

Dengan menerapkan hasil konvolusi  $S(1,1) = -0,1667$  ke dalam fungsi ReLU, diperoleh:

$$f(-0,1667) = \max(0, -0,1667) = 0,0000$$

Hasil tersebut menunjukkan bahwa nilai aktivasi negatif akan dipetakan menjadi nol, sehingga neuron pada posisi tersebut tidak memberikan kontribusi terhadap lapisan berikutnya.

c. Max Pooling ( $2 \times 2$ , Stride 2)

Lapisan Max Pooling berfungsi mereduksi dimensi spasial feature map dengan mengambil nilai maksimum dari setiap wilayah (region) yang ditinjau. Misalkan hasil feature map sebelum pooling berupa matriks berukuran  $2 \times 2$  sebagai berikut:

$$M = \begin{bmatrix} 0,85 & 0,42 \\ 0,12 & 0,67 \end{bmatrix}$$

$$\text{MaxPool}(M) = \max(0,85; 0,42; 0,12; 0,67) = 0,85$$

Nilai maksimum 0,85 tersebut dipilih sebagai representasi dari keempat nilai pada region matriks  $M$ , sehingga dimensi spasial feature map berkurang tanpa menghilangkan informasi fitur yang paling dominan.

d. Global Average Pooling (GAP)

Lapisan Global Average Pooling digunakan untuk meringkas feature map tiga dimensi menjadi sebuah vektor satu dimensi dengan cara menghitung nilai rata-rata dari seluruh elemen pada setiap kanal. Misalkan suatu feature map berukuran  $2 \times 2$

pada satu kanal memiliki nilai [0,8; 0,6; 0,4; 0,2], maka nilai keluaran GAP dihitung sebagai berikut:

$$GAP = (0,8+0,6+0,4+0,2) / 4 = 2,0 / 4 = 0,50$$

e. Probabilitas Softmax

Fungsi aktivasi Softmax digunakan pada lapisan keluaran untuk mengubah nilai logit mentah menjadi distribusi probabilitas dari keempat kelas target. Misalkan nilai logit sebelum Softmax untuk keempat kelas (moler, non\_bawang, sehat, trotol) adalah  $z = [0,5; -1,2; 2,1; 0,1]$ , maka terlebih dahulu dihitung nilai eksponensial dari setiap logit:

$$e^{0,5} \approx 1,6487 \quad e^{-1,2} \approx 0,3012 \quad e^{2,1} \approx 8,1662 \quad e^{0,1} \approx 1,1052$$

Jumlah keseluruhan nilai eksponensial dihitung sebagai berikut:

$$\sum e^{z_j} = 1,6487 + 0,3012 + 8,1662 + 1,1052 = 11,2213$$

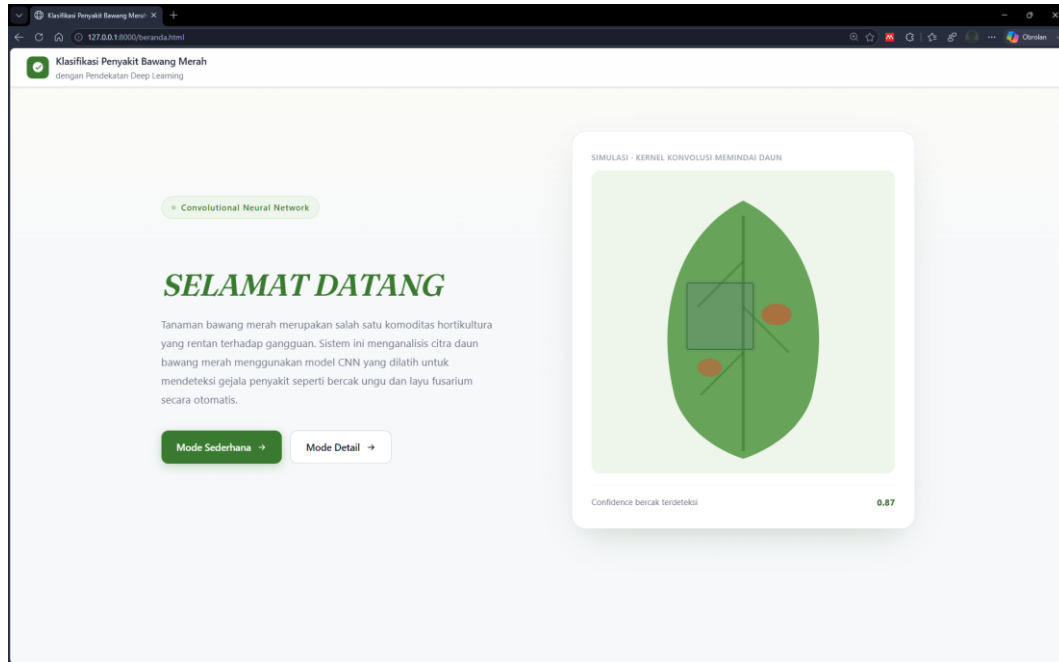
Sehingga probabilitas untuk kelas ketiga (sehat) dihitung dengan membagi nilai eksponensial kelas tersebut dengan jumlah keseluruhan nilai eksponensial:

$$P(\text{sehat}) = 8,1662 / 11,2213 \approx 0,7277 \text{ (72,77\%)}$$

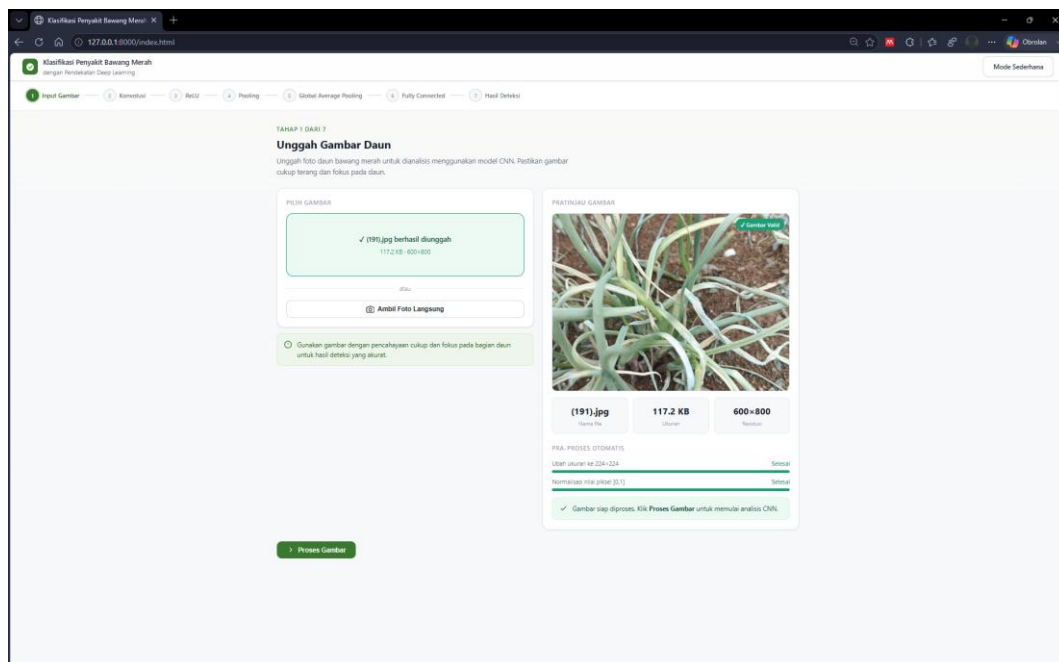
Nilai probabilitas 72,77% tersebut merupakan nilai tertinggi di antara keempat kelas, sehingga model akan mengklasifikasikan citra masukan tersebut ke dalam kelas sehat dengan tingkat kepercayaan (confidence score) sebesar 72,77%.

## E. Implementasi Sistem

### 1. Halaman Beranda



### 2. Halaman Input

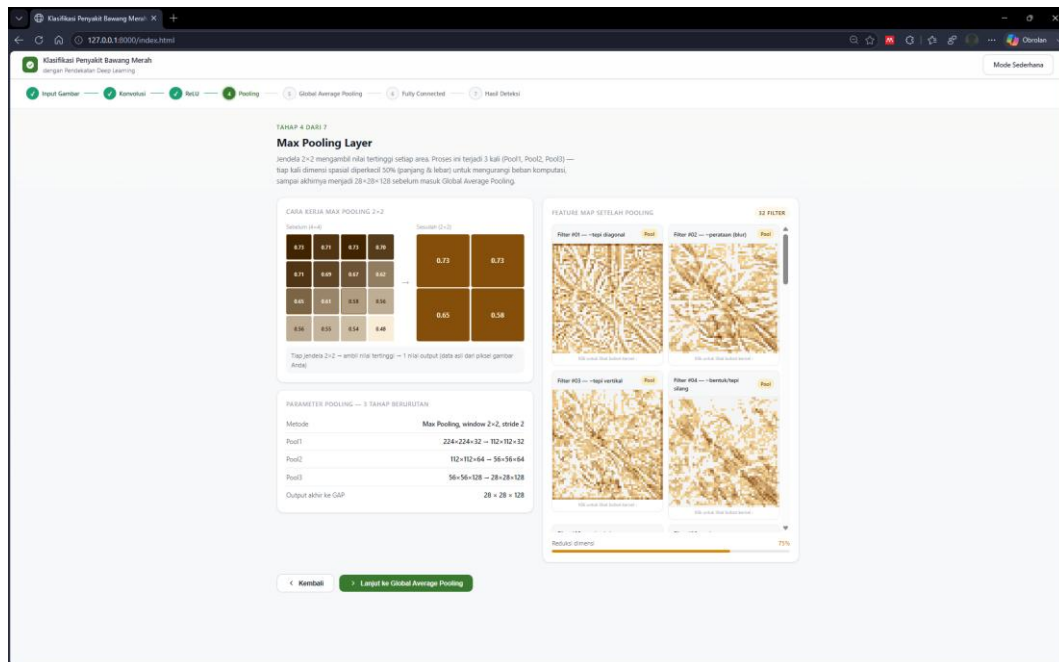




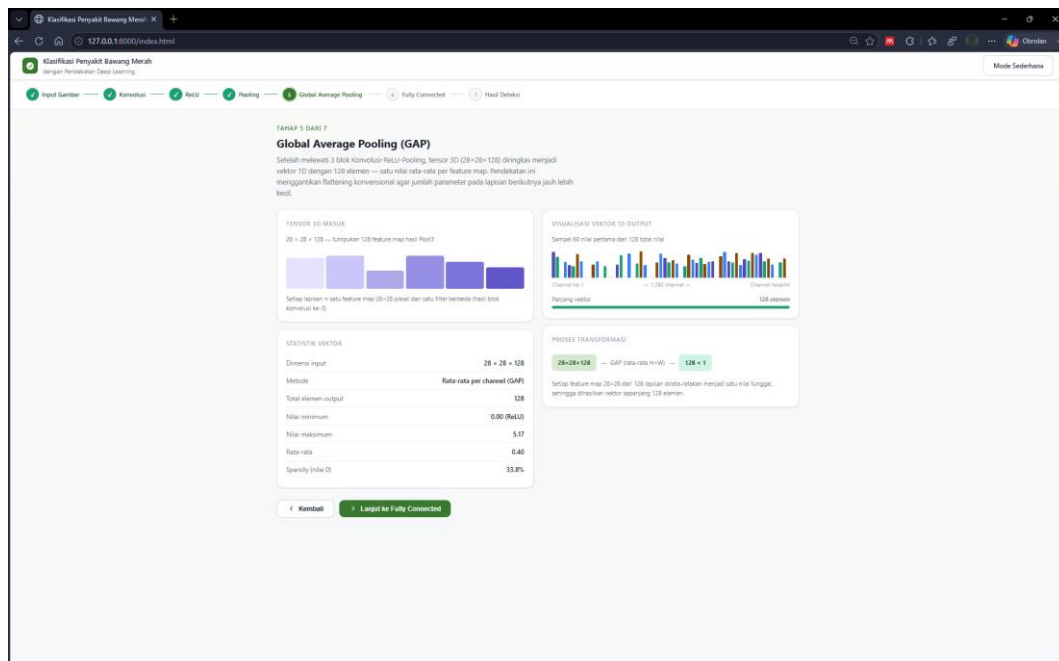
### 3. Halaman Konvolusi

### 4. Halaman ReLU

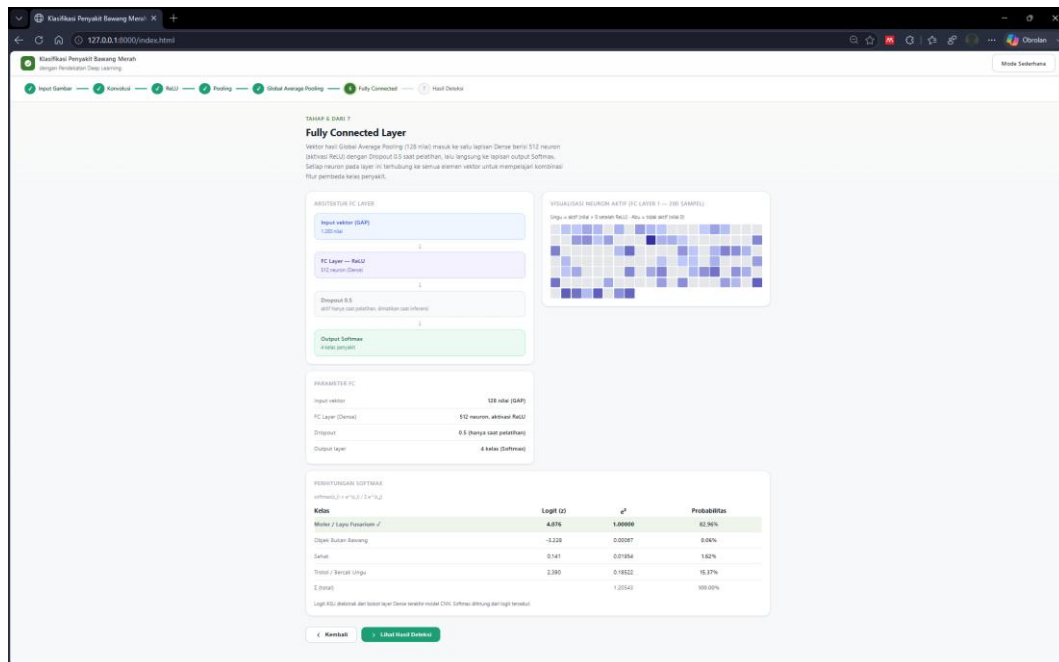
## 5. Halaman Pooling



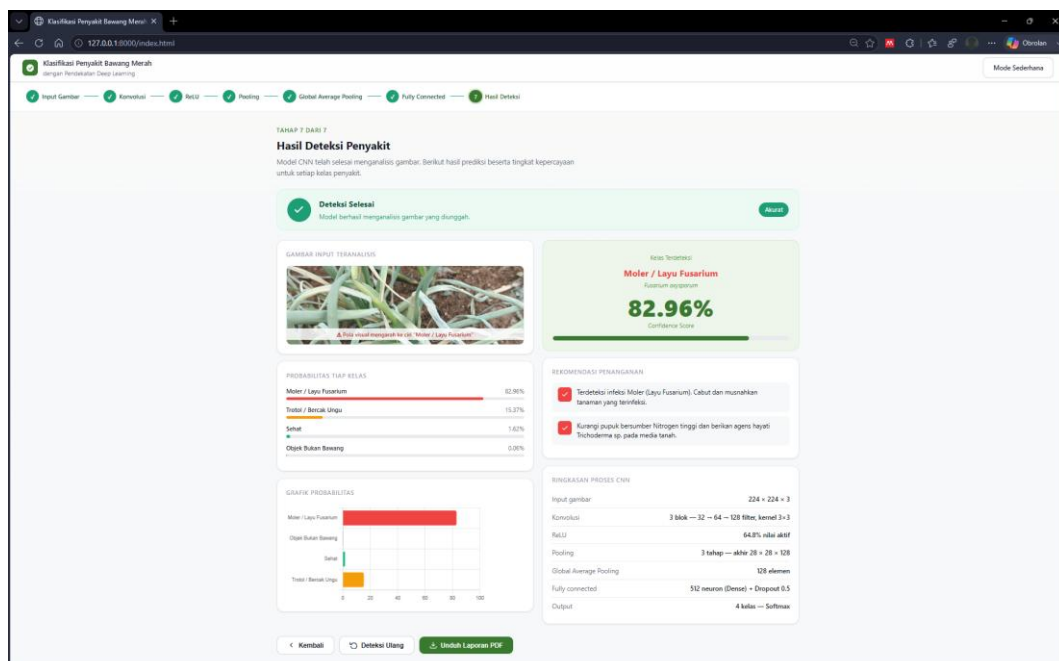
## 6. Halaman Global Average Pooling



## 7. Halaman Fully Connected Layer



## 8. Halaman Hasil Deteksi



## **F. Pengujian Sistem**

Pengujian sistem dilakukan untuk memastikan bahwa seluruh fungsi pada aplikasi web klasifikasi penyakit bawang merah berjalan sesuai dengan rancangan yang telah ditetapkan, baik dari sisi fungsionalitas antarmuka pengguna maupun dari sisi performa model Convolutional Neural Network (CNN) yang telah dilatih. Pengujian pada bagian ini terbagi menjadi dua tahap, yaitu pengujian fungsionalitas sistem menggunakan metode Black-Box Testing dan rekapitulasi hasil pelatihan model secara keseluruhan.

### **1. Pengujian Black Box**

Pengujian fungsionalitas antarmuka web dilakukan menggunakan metode Black-Box Testing, yaitu teknik pengujian perangkat lunak yang berfokus pada verifikasi keluaran sistem berdasarkan masukan yang diberikan tanpa perlu meninjau struktur kode program secara internal. Metode ini dipilih karena bertujuan untuk memastikan bahwa sistem memberikan respons yang sesuai terhadap berbagai skenario interaksi pengguna, mulai dari proses pengunggahan citra hingga proses penyajian hasil klasifikasi. Rincian skenario pengujian beserta hasilnya disajikan pada Tabel 4.4.

| No | Skenario Pengujian             | Masukan (Input)            | Hasil yang Diharapkan                                             | Hasil Pengujian   | Status |
|----|--------------------------------|----------------------------|-------------------------------------------------------------------|-------------------|--------|
| 1  | Pengunggahan gambar valid      | Berkas .jpg / .png         | Gambar diterima, pratinjau tampil, proses CNN berjalan            | Sesuai ekspektasi | Valid  |
| 2  | Pengunggahan berkas non-gambar | Berkas .pdf / .docx        | Sistem menolak berkas & menampilkan pesan penolakan               | Sesuai ekspektasi | Valid  |
| 3  | Tangkapan kamera langsung      | Izin kamera diberikan      | Foto berhasil ditangkap dan diproses ke pipeline                  | Sesuai ekspektasi | Valid  |
| 4  | Navigasi visualisasi CNN       | Klik tombol Lanjut/Kembali | Tampilan berpindah antar step konvolusi, ReLU, pooling, dll.      | Sesuai ekspektasi | Valid  |
| 5  | Tampilan hasil klasifikasi     | Citra daun bercak ungu     | Menampilkan label Trotol & score probabilitas beserta rekomendasi | Sesuai ekspektasi | Valid  |
| 6  | Simpan & unduh laporan         | Klik Unduh Laporan PDF     | Berkas laporan PDF terunduh berisi rincian hasil deteksi          | Sesuai ekspektasi | Valid  |

Berdasarkan hasil pengujian pada Tabel 4.4, keenam skenario pengujian yang mencakup validasi format berkas, penangkapan citra melalui kamera, navigasi antar tahapan visualisasi pipeline CNN, penyajian hasil klasifikasi beserta rekomendasi penanganan, hingga fitur pengunduhan laporan dalam format PDF, seluruhnya menunjukkan hasil yang sesuai dengan ekspektasi (dinyatakan "Valid"). Dengan demikian, dapat disimpulkan bahwa sistem telah berfungsi dengan baik dan siap digunakan oleh pengguna akhir, khususnya petani bawang merah, tanpa ditemukan kendala fungsional yang berarti pada seluruh alur interaksi yang diuji.

## 2. Hasil Pelatihan Model

Selain pengujian fungsionalitas antarmuka, dilakukan pula rekapitulasi terhadap hasil pelatihan dan pengujian model CNN MobileNetV2 secara keseluruhan untuk memberikan gambaran ringkas mengenai performa akhir model yang telah dikembangkan. Ringkasan tersebut mencakup komposisi dataset yang digunakan

serta nilai-nilai metrik evaluasi yang telah dibahas pada sub-bab sebelumnya, sebagaimana disajikan pada Tabel 4.5.

| Metrik Evaluasi                     | Nilai Hasil Pelatihan / Pengujian          |
|-------------------------------------|--------------------------------------------|
| Total Dataset                       | 2.000 Citra                                |
| Pembagian Data (Train / Val / Test) | 70% (1.400) / 15% (300) / 15% (300)        |
| Jumlah Kelas Target                 | 4 Kelas (moler, non_bawang, sehat, trotol) |
| Test Accuracy (Data Uji 300 Sampel) | 94,33%                                     |
| Macro Average Precision             | 94,87%                                     |
| Macro Average Recall                | 94,58%                                     |
| Macro Average F1-Score              | 94,71%                                     |
| Weighted Average F1-Score           | 94,33%                                     |

Tabel 4.5 menunjukkan bahwa model CNN MobileNetV2 yang dilatih menggunakan 1.400 citra data training dari total 2.000 citra dataset, mampu mencapai Test Accuracy sebesar 94,33% pada 300 data uji yang independen. Nilai Macro Average Precision, Recall, dan F1-Score yang masing-masing berada di atas 94% turut menegaskan bahwa performa model tergolong konsisten dan tidak bias terhadap kelas tertentu, meskipun jumlah sampel antar kelas pada dataset tidak sepenuhnya seimbang (kelas Sehat dan Trotol memiliki jumlah sampel lebih banyak dibandingkan kelas Moler dan Bukan Bawang). Secara keseluruhan, hasil pengujian fungsional maupun kuantitatif ini membuktikan bahwa sistem klasifikasi penyakit bawang merah berbasis CNN MobileNetV2 telah memenuhi kriteria kelayakan baik dari aspek keandalan fungsi (functional reliability) maupun akurasi prediksi (predictive accuracy), sehingga layak untuk diimplementasikan sebagai alat bantu diagnosis dini bagi petani bawang merah.

## **BAB V**

## **PENUTUP**

## DAFTAR PUSTAKA

- Allam, S. F., & Wibowo, A. P. (2023). 22.-sayyidan-fahd-allam. *Jurnal Komputer dan Informatika*, 5.
- Asrianda, A., Aidilof, H. A. K., & Pangestu, Y. (2021). Machine Learning for Detection of Palm Oil Leaf Disease Visually using Convolutional Neural Network Algorithm. *JOURNAL OF INFORMATICS AND TELECOMMUNICATION ENGINEERING*, 4(2), 286–293. <https://doi.org/10.31289/jite.v4i2.4185>
- Azurah, D., Darwis, H., Nur Hayati, L., & Artikel Abstrak, I. (2023). Klasifikasi Penyakit Bawang Merah Menggunakan Naïve Bayes dan Convolutional Neural Network. *Indonesian Journal of Computer Science Attribution*, 12(4), 1932.
- Bednarz, B., & Miłosz, M. (2025). *Benchmarking the performance of Python web frameworks Analiza porównawcza wydajności webowych szkieletów programistycznych w języku Python*.
- Deden, D., & Wijaya, W. (2023). Efektivitas Agen Hayati (*Rhodopseudomonas palustris*) untuk Mengendalikan Penyakit Bercak Daun (*Alternaria porri*) pada Tanaman Bawang Merah (*Allium ascalonicum* L.). *AGROSCRIPT: Journal of Applied Agricultural Sciences*, 5(2), 92–100. <https://doi.org/10.36423/agroscript.v5i2.1212>
- Gedam, M. N., & Meshram, B. B. (2023). Jijabai Technological Institute (VJTI). Dalam *IJACSA) International Journal of Advanced Computer Science and Applications* (Vol. 14, Nomor 6). [www.ijacsa.thesai.org](http://www.ijacsa.thesai.org)
- Idkowiak, J., Dehairs, J., Schwarzerová, J., Olešová, D., Truong, J. X. M., Kvasnička, A., Eftychiou, M., Cools, R., Spotbeen, X., Jirásko, R., Veseli, V., Giampà, M., de Laat, V., Butler, L. M., Weckwerth, W., Friedecký, D., Demeulemeester, J., Hron, K., Swinnen, J. V., & Holčapek, M. (2025). Best practices and tools in R and Python for statistical processing and visualization of lipidomics and metabolomics data. Dalam *Nature Communications* (Vol. 16, Nomor 1). Nature Research. <https://doi.org/10.1038/s41467-025-63751-1>
- Joeniarti, E., Hadiwiyono, Septariani, D., & Poromarto, S. H. (2024). *Seminar Nasional Pengabdian dan CSR Ke-4 Fakultas Pertanian Universitas Sebelas Maret, Surakarta Tahun 2024* (Vol. 4, Nomor 1).
- Juliansyah, S., & Laksito, A. D. (2021). Klasifikasi Citra Buah Pir Menggunakan Convolutional Neural Networks. *Jurnal Telekomunikasi*



*dan Komputer*, 11(1), 65–72.

<https://doi.org/10.22441/incomtech.v10i2.10185>

- Koukaras, P., & Tjortjis, C. (2025). Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices. Dalam *AI (Switzerland)* (Vol. 6, Nomor 10). Multidisciplinary Digital Publishing Institute (MDPI). <https://doi.org/10.3390/ai6100257>
- Mubarak, A., Marlina, M., & Hasnawati, H. (2025). Rancang Bangun Aplikasi Android Untuk Monitoring Pasokan Bawang Merah Di Kabupaten Enrekang. *Jurnal Ilmiah Sistem Informasi dan Teknik Informatika (JISTI)*, 8(2), 175–181. <https://doi.org/10.57093/jisti.v8i2.318>
- Nematzadeh, S., Anka, F., Ciftci, F., Ayanoglu, K. Y. U., Özarslan, A. C., & Oncu, E. (2026). Bridging engineering and neuro-oncology: a scalable FastAPI-deployed CNN framework for real-time explainable brain tumor diagnosis. *Frontiers in Neuroscience*, 20. <https://doi.org/10.3389/fnins.2026.1772429>
- Nur, Y. S. R., Burhanuddin, A., Aldo, D., & Lelisa Army, W. (2022). Sistem Pakar Deteksi Penyakit Bawang Merah dengan Metode Case Based Reasoning. *JURNAL MEDIA INFORMATIKA BUDIDARMA*, 6(3), 1356. <https://doi.org/10.30865/mib.v6i3.4180>
- Pamungkas, D. P., & Amrulloh, M. F. (2025). ANALISIS HASIL KLASIFIKASI PENYAKIT DAUN BAWANG MERAH MENGGUNAKAN CNN ARSITEKTUR EXCEPTION. *JUPI (Jurnal Ilmiah Penelitian dan Pembelajaran Informatika)*, 10(1), 359–366. <https://doi.org/10.29100/jupi.v10i1.5875>
- Permata Sari, A. (2020). RANCANG BANGUN SISTEM INFORMASI PENGELOLAAN TALENT FILM BERBASIS APLIKASI WEB. *Jurnal Informatika Terpadu*, 6(1), 29–37. <https://journal.nurulfikri.ac.id/index.php/JIT>
- Pongu Lima Manang, A., & dan Astri Sumiati, S. (2024). Review Penyakit Layu Fusarium Dan Pengendaliannya Pada Bawang Merah. Dalam *Alium Escalonicum L.*. *Jurnal Buana Sains* (Vol. 24, Nomor 3).
- Praniffa, A. C., Syahri, A., Sandes, F., Fariha, U., & Giansyah, Q. A. (2023). PENGUJIAN BLACK BOX DAN WHITE BOX SISTEM INFORMASI PARKIR BERBASIS WEB BLACK BOX AND WHITE BOX TESTING OF WEB-BASED PARKING INFORMATION SYSTEM. Dalam *Jurnal Testing dan Implementasi Sistem Informasi* (Vol. 1, Nomor 1).

- Qin, Y. M., Tu, Y. H., Li, T., Ni, Y., Wang, R. F., & Wang, H. (2025). Deep Learning for Sustainable Agriculture: A Systematic Review on Applications in Lettuce Cultivation. Dalam *Sustainability (Switzerland)* (Vol. 17, Nomor 7). Multidisciplinary Digital Publishing Institute (MDPI). <https://doi.org/10.3390/su17073190>
- Rijal, M., Yani, A. M., & Rahman, A. (2024). DETEKSI CITRA DAUN UNTUK KLASIFIKASI PENYAKIT PADI MENGGUNAKAN PENDEKATAN DEEP LEARNING DENGAN MODEL CNN. *Jurnal Teknologi Terpadu*, 10(1), 56–62.
- Riqza Ardiansyah, A., & Putra Pamungkas, D. (2024). KLASIFIKASI MENGGUNAKAN METODE SUPPORT VECTOR MACHINE UNTUK MENDETEKSI PENYAKIT TANAMAN BAWANG MERAH. *Jurnal Nusantara Of Engineering*, 7. <https://ojs.unpkediri.ac.id/index.php/noe>
- Schmidt, A. F., Hukerikar, N., Finan, C., & van Vugt, M. (2026). Effective visualisation of biomedical data using plot-misc . *Bioinformatics Advances*. <https://doi.org/10.1093/bioadv/vbag184>
- Sholihah, S. N. (2024). *KOMODITAS PERTANIAN SUBSEKTOR HORTIKULTURA BAWANG MERAH*. Pusat Data dan Sistem Informasi Pertanian Kementerian Pertanian.
- Singh, V., Sahana, S. K., & Bhattacharjee, V. (2025). A novel CNN-GRU-LSTM based deep learning model for accurate traffic prediction. *Discover Computing*, 28(1). <https://doi.org/10.1007/s10791-025-09526-0>
- Smrti, N., Andisana, P., Rahayu, N., Adnan, & Juliantara, P. (2023). *Flowgorithm Sebagai Penunjang Pembelajaran Algoritma dan Pemrograman*.
- Susandi, Y. N. K., Sualang, D. S., & Paruntu, M. H. B. (t.t.). *ANTAGONISME Trichoderma sp. TERHADAP Alternaria porri PATOGEN PENYAKIT BERCAK UNGU TANAMAN BAWANG MERAH PADA BEBERAPA MEDIA The Antagonism of Trichoderma sp. Against Alternaria porri a Pathogen Causing Purple Blotch Disease of Shallot on Several Media*.